UNIT-5 (GRAPHS & HASHING)

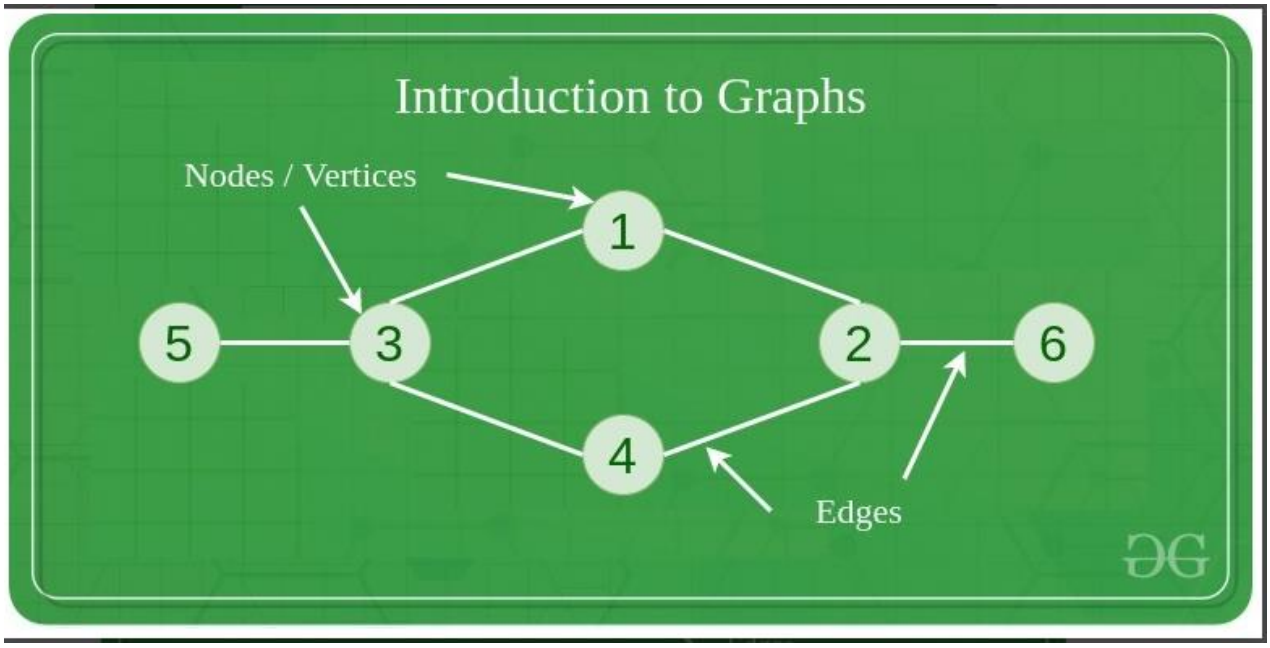
BASIC DEFINITION AND TERMINOLOGY – REPRESENTATION OF GRAPH

LECTURE NO:-33

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The graph is denoted by **G(V, E).**

Components of a Graph

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labelled/unlabelled.

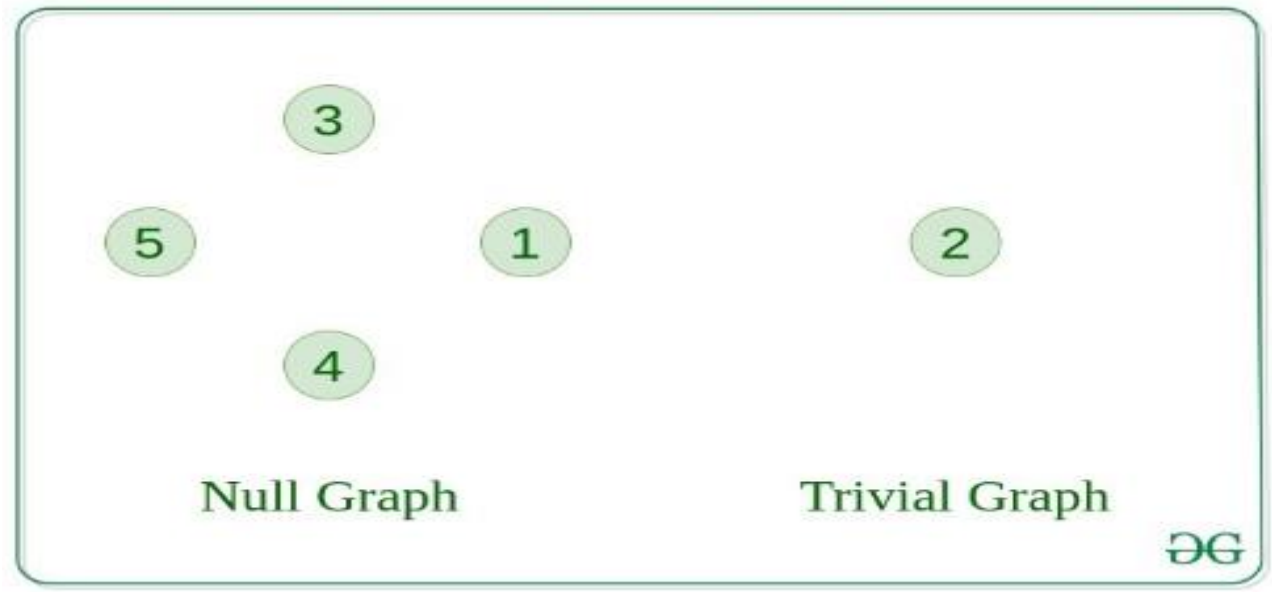


Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

Types Of Graph

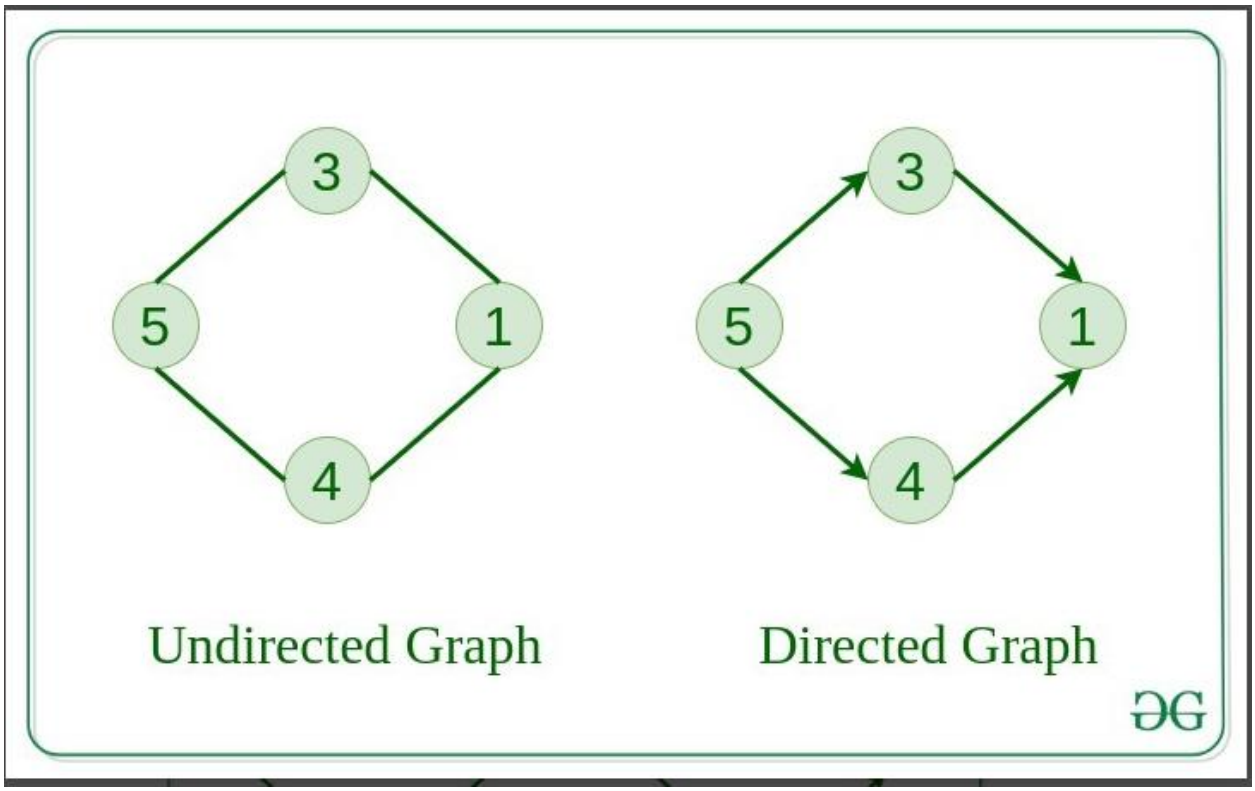
1. Null Graph: A graph is known as a null graph if there are no edges in the graph.

2. Trivial Graph: Graph having only a single vertex, it is also the smallest graph possible.



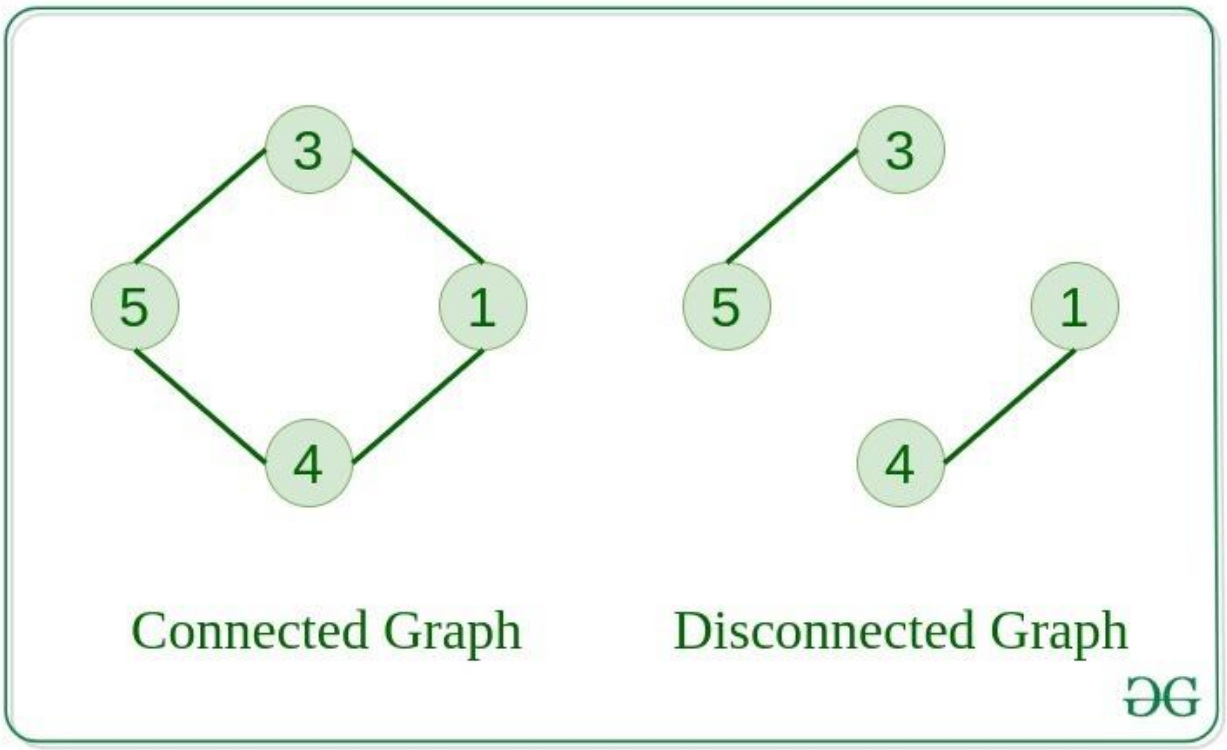
3. Undirected Graph : A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. Directed Graph : A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



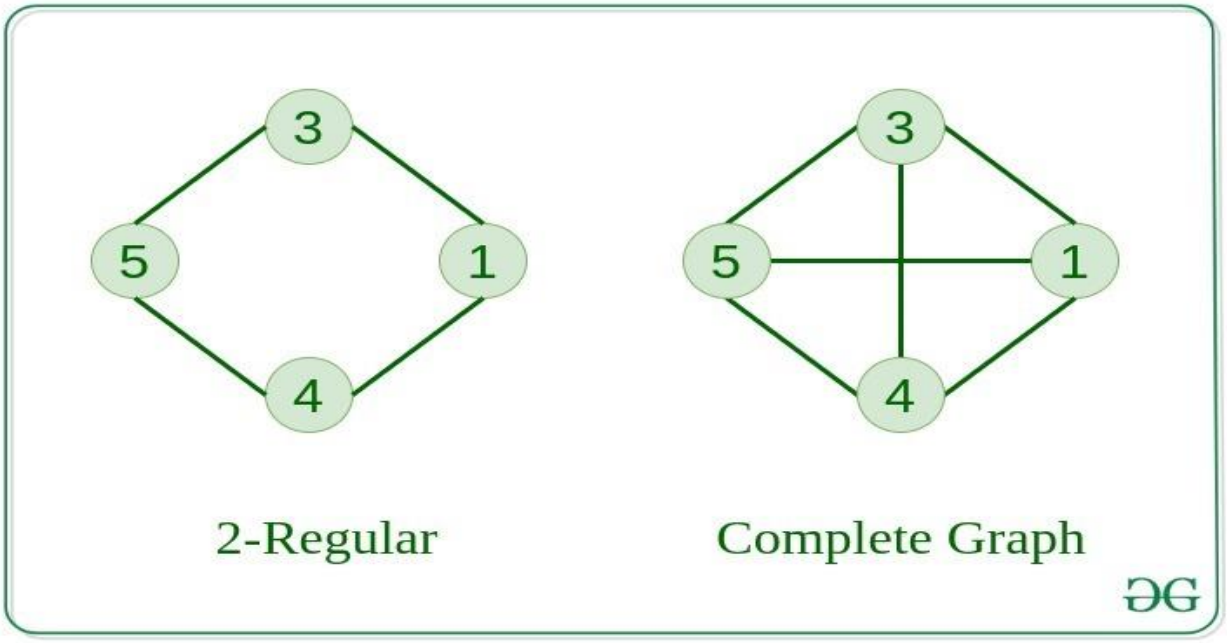
5. Connected Graph: The graph in which from one node we can visit any other node in the graph is known as a connected graph.

6. Disconnected Graph: The graph in which at least one node is not reachable from a node is known as a disconnected graph.



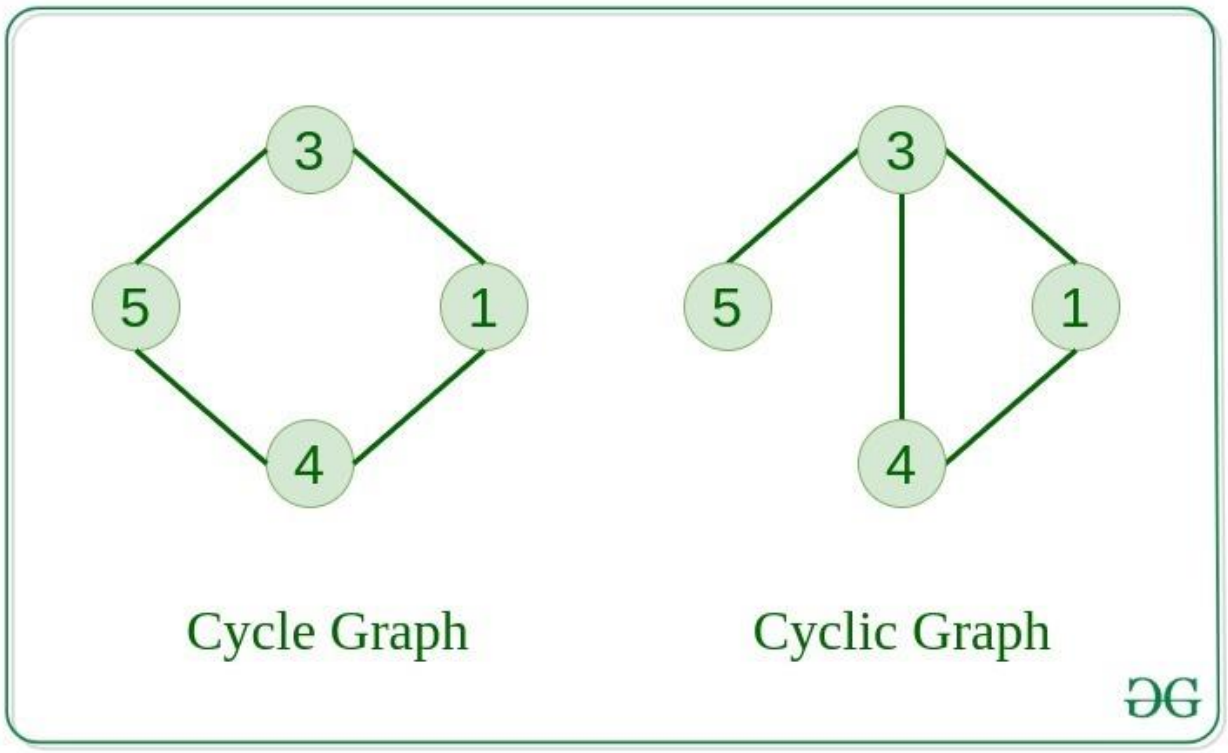
7. Regular Graph: The graph in which the degree of every vertex is equal to K is called K regular graph.

8. Complete Graph: The graph in which from each node there is an edge to each other node.



9. Cycle Graph: The graph in which the graph is a cycle in itself, the degree of each vertex is 2.

10. Cyclic Graph: A graph containing at least one cycle is known as a Cyclic graph.



11. Weighted Graph

- A graph in which the edges are already specified with suitable weight is known as a weighted graph.
- Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.

Graph Representations

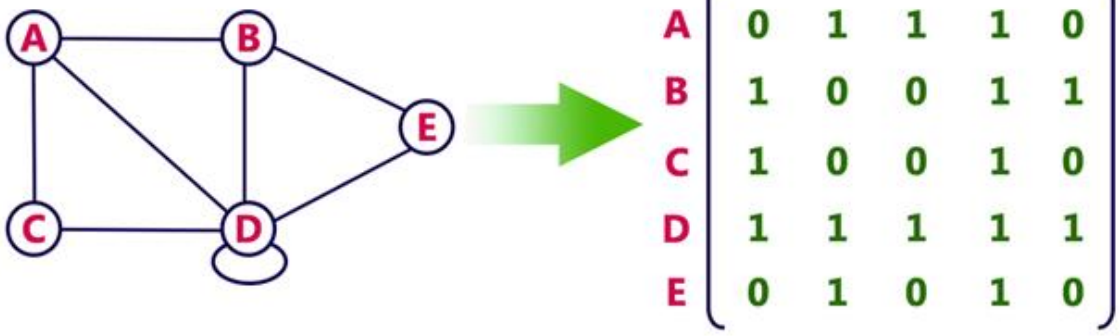
There are different ways to optimally represent a graph, depending on the density of its edges, type of operations to be performed and ease of use.

1. Adjacency Matrix

- Adjacency matrix is a sequential representation.
- It is used to represent which nodes are adjacent to each other. i.e. is there any edge connecting nodes to a graph.

- if there is any weighted graph then instead of 1s and 0s, we can store the weight of the edge.

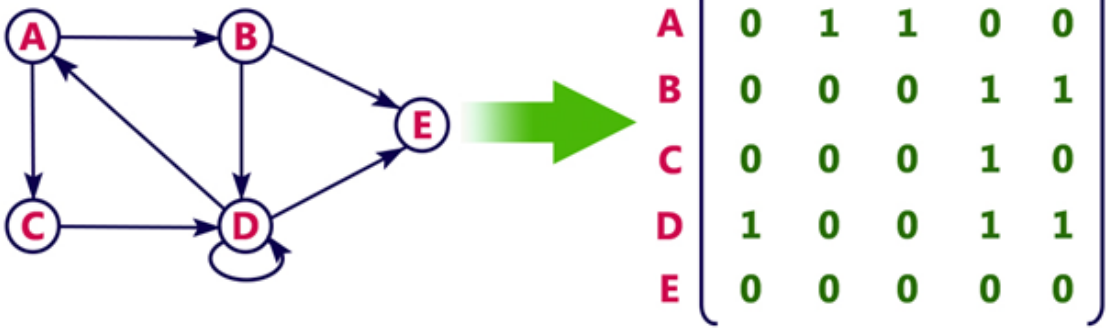
Undirected graph representation



	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	1	1
E	0	1	0	1	0

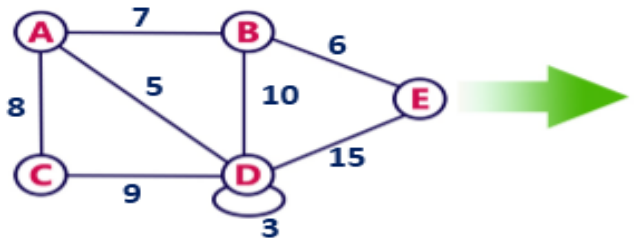
Directed graph represenation

See the directed graph representation:



	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	1
C	0	0	0	1	0
D	1	0	0	1	1
E	0	0	0	0	0

Undirected Weighted Graph Representation



	A	B	C	D	E
A	0	7	8	5	0
B	7	0	0	10	6
C	8	0	0	9	0
D	5	10	9	3	15
E	0	6	0	15	0

Pros: Representation is easier to implement and follow.

Cons: It takes a lot of space and time to visit all the neighbours of a vertex, we have to traverse all the vertices in the graph, which takes quite some time.

2. Incidence Matrix

In Incidence matrix representation, graph can be represented using a matrix of size:

Total number of vertices by total number of edges.

It means if a graph has 4 vertices and 6 edges, then it can be represented using a matrix of 4X6 class. In this matrix, columns represent edges and rows represent vertices.

This matrix is filled with either 0 or 1 or -1. Where,

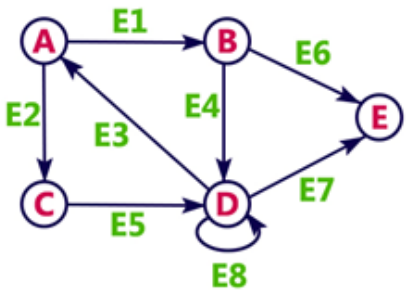
0 is used to represent row edge which is not connected to column vertex.

1 is used to represent row edge which is connected as outgoing edge to column vertex.

-1 is used to represent row edge which is connected as incoming edge to column vertex.

Example

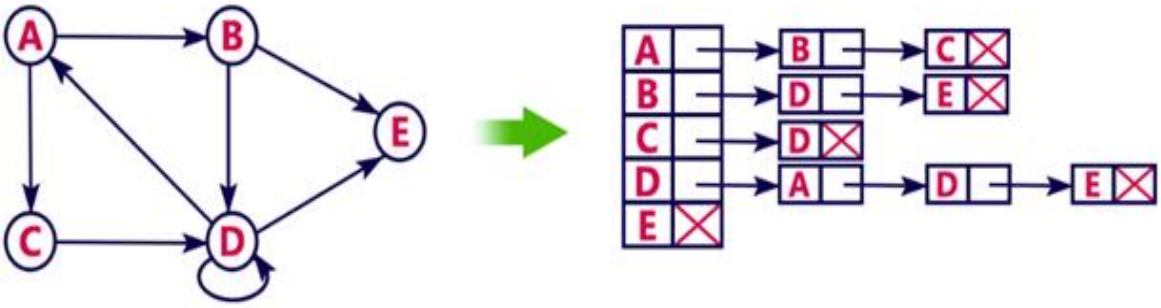
Consider the following directed graph representation.



	E1	E2	E3	E4	E5	E6	E7	E8
A	1	1	-1	0	0	0	0	0
B	-1	0	0	1	0	1	0	0
C	0	-1	0	0	1	0	0	0
D	0	0	1	-1	-1	0	1	1
E	0	0	0	0	0	-1	-1	0

3. Adjacency List

- Adjacency list is a linked representation.
- In this representation, for each vertex in the graph, we maintain the list of its neighbors. It means, every vertex of the graph contains list of its adjacent vertices.



Pros:

- Adjacency list saves lot of space.
- We can easily insert or delete as we use linked list.
- Such kind of representation is easy to follow and clearly shows the adjacent nodes of node.

Cons:

- The adjacency list allows testing whether two vertices are adjacent to each other but it is slower to support this operation.

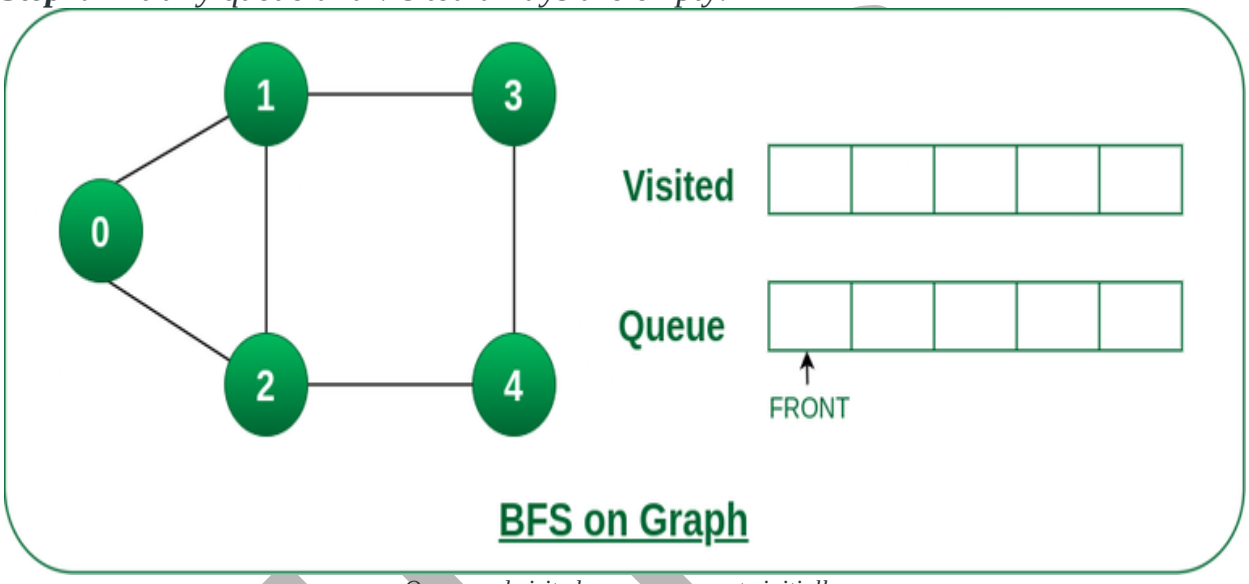
UNIT-5 (GRAPHS & HASHING)

GRAPH TRAVERSAL: BREADTH FIRST SEARCH (BFS)

LECTURE NO:-34

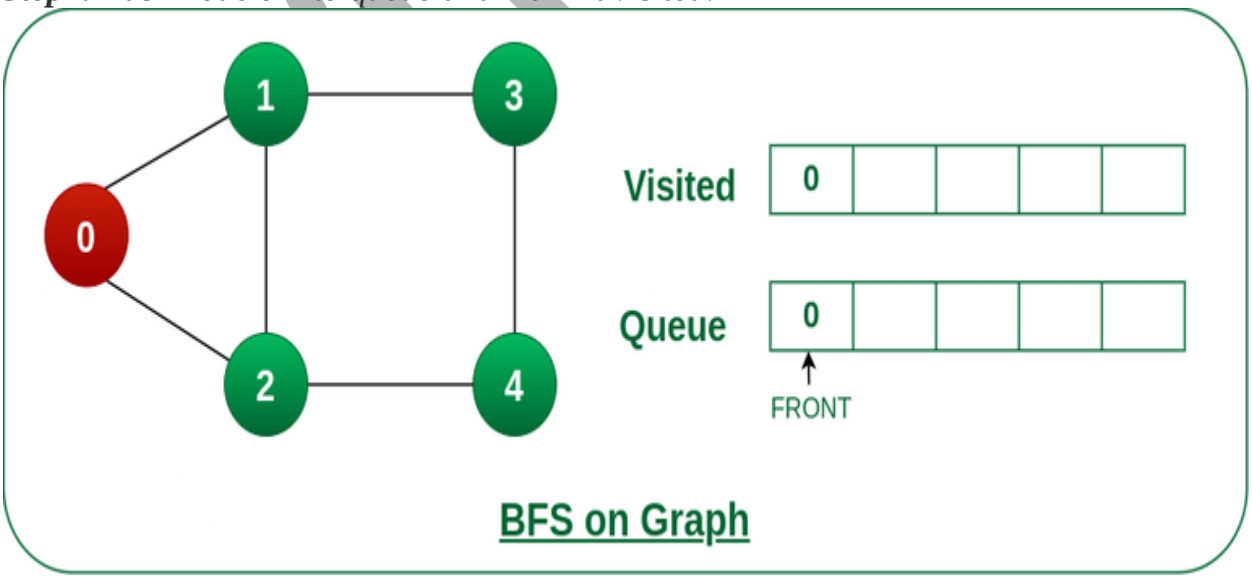
*The **Breadth First Search (BFS)** algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before moving on to the nodes at the next depth level.*

***Step1:** Initially queue and visited arrays are empty.*



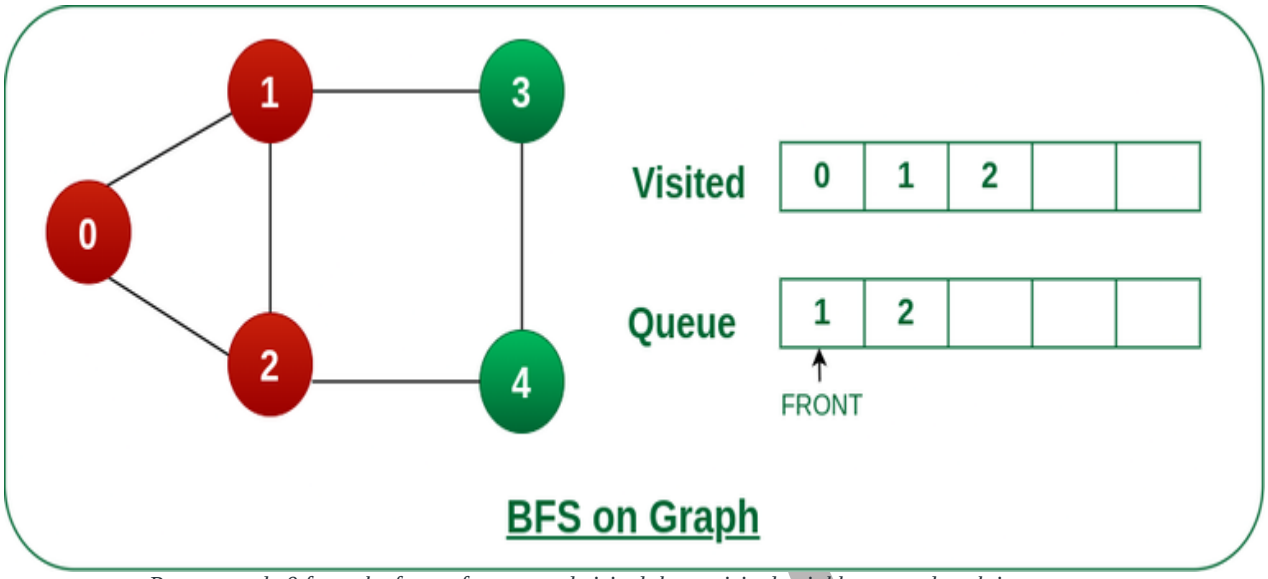
Queue and visited arrays are empty initially.

***Step2:** Push node 0 into queue and mark it visited.*



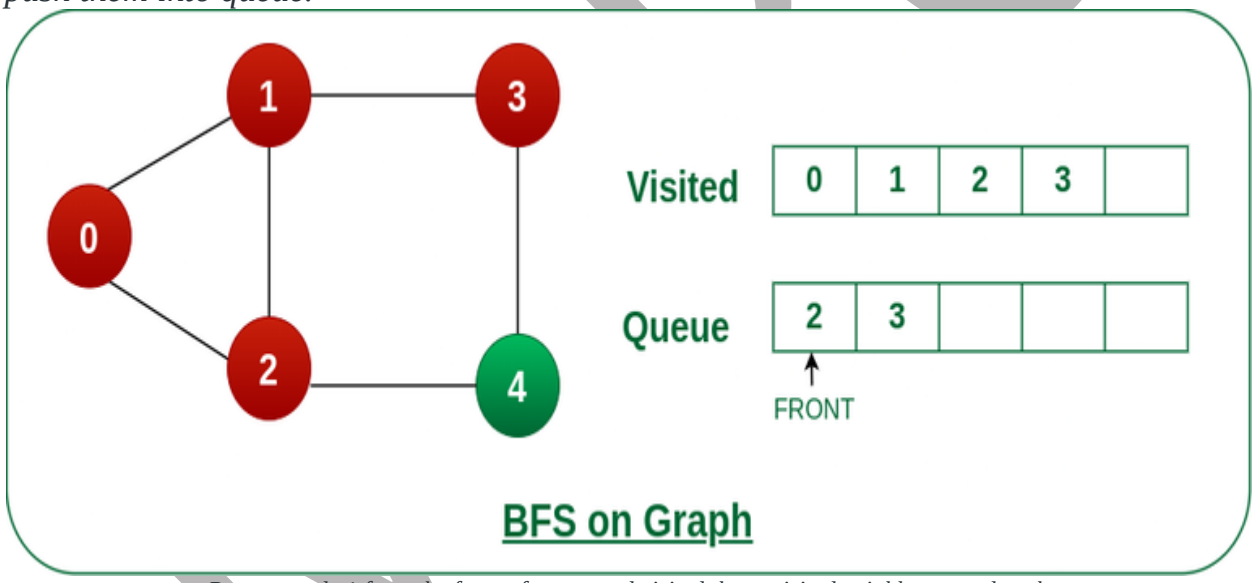
Push node 0 into queue and mark it visited.

***Step 3:** Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.*



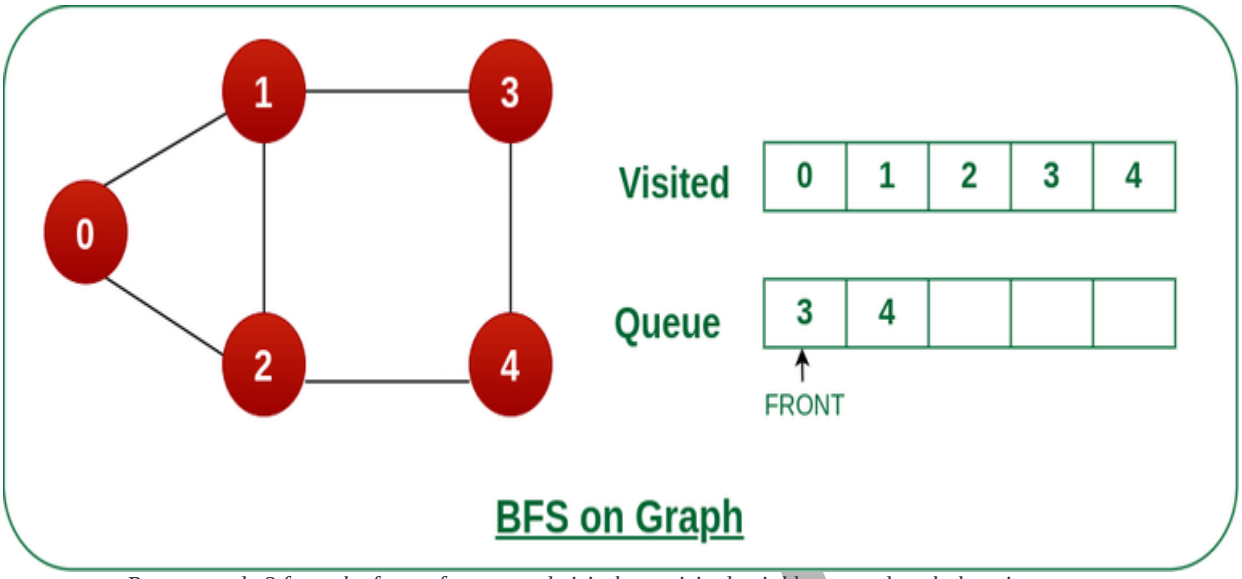
Remove node 0 from the front of queue and visited the unvisited neighbours and push into queue.

Step 4: *Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.*



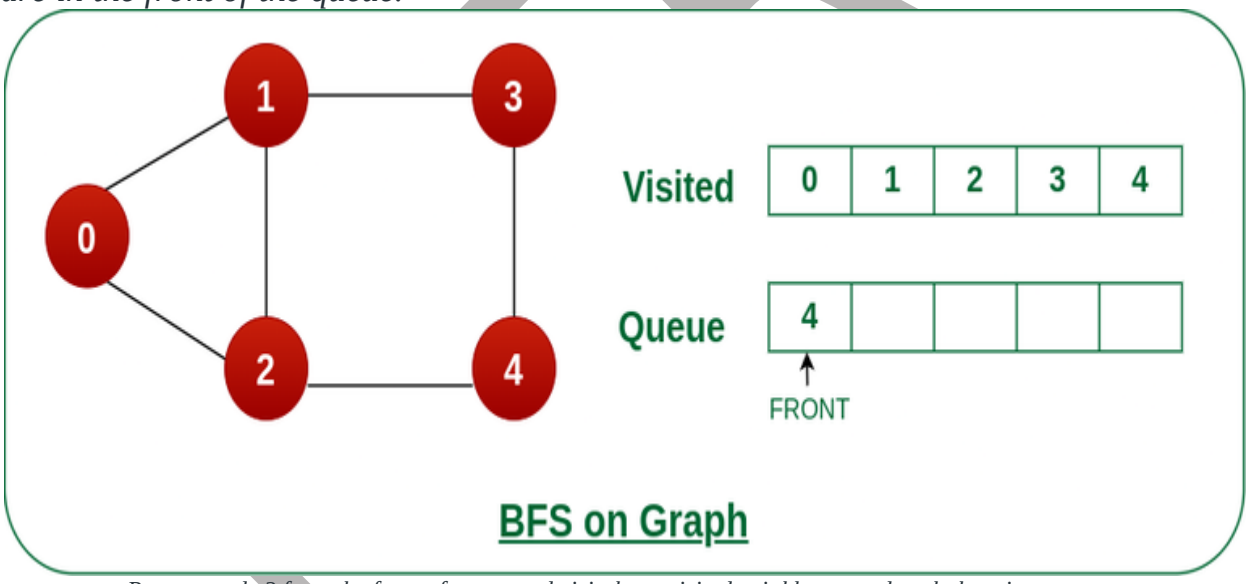
Remove node 1 from the front of queue and visited the unvisited neighbours and push

Step 5: *Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.*



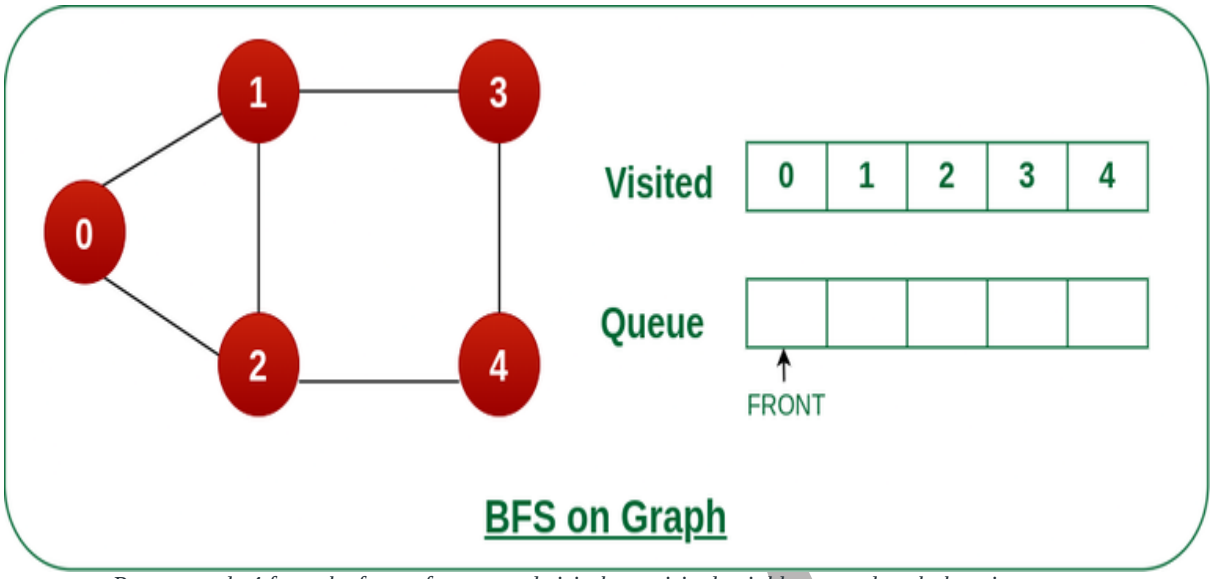
Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.

Step 6: *Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.*
As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

Steps 7: *Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.*
As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.

MITRC

UNIT-5 (GRAPHS & HASHING)

DEPTH FIRST SEARCH (DFS)

LECTURE NO:-35

Algorithm

Step 1: SET STATUS = 1 (ready state) for each node in G

Step 2: Push the starting node A on the stack and set its STATUS = 2 (waiting state)

Step 3: Repeat Steps 4 and 5 until STACK is empty

Step 4: Pop the top node N. Process it and set its STATUS = 3 (processed state)

Step 5: Push on the stack all the neighbors of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)

[END OF LOOP]

Notes Prepared By: - Arvind Sharma (Assistant Professor, Department of CSE, MITRC Alwar)

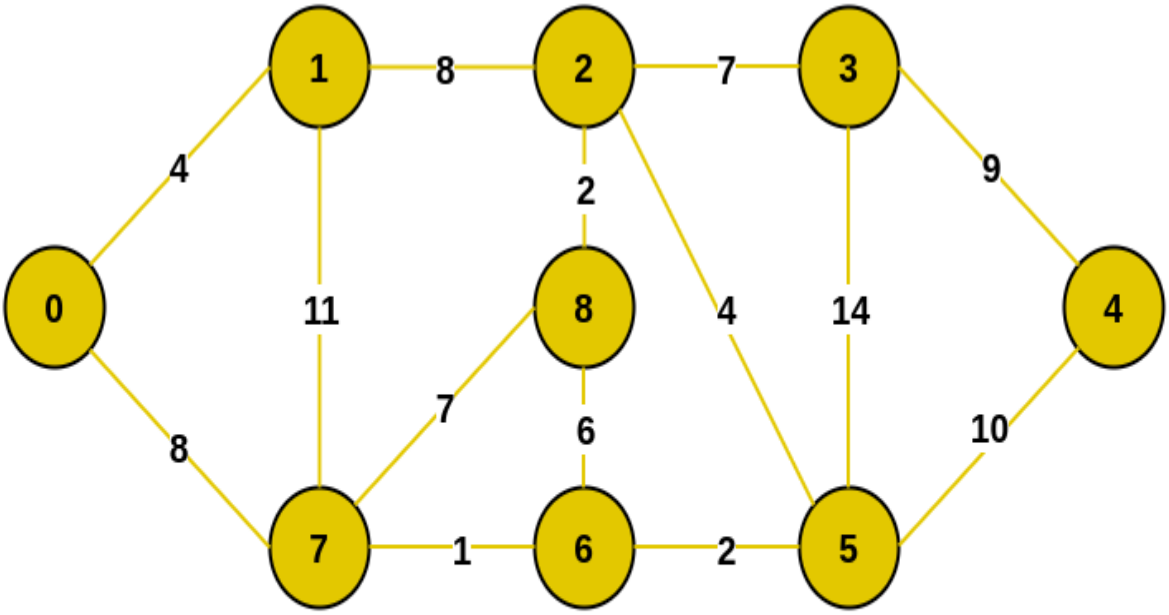
UNIT-5 (GRAPHS & HASHING)

MINIMUM SPANNING TREE : PRIM'S

LECTURE NO:-36

Step 1: *Determine an arbitrary vertex as the starting vertex of the MST.*
Step 2: *Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).*
Step 3: *Find edges connecting any tree vertex with the fringe vertices.*
Step 4: *Find the minimum among these edges.*
Step 5: *Add the chosen edge to the MST if it does not form any cycle.*
Step 6: *Return the MST and exit*

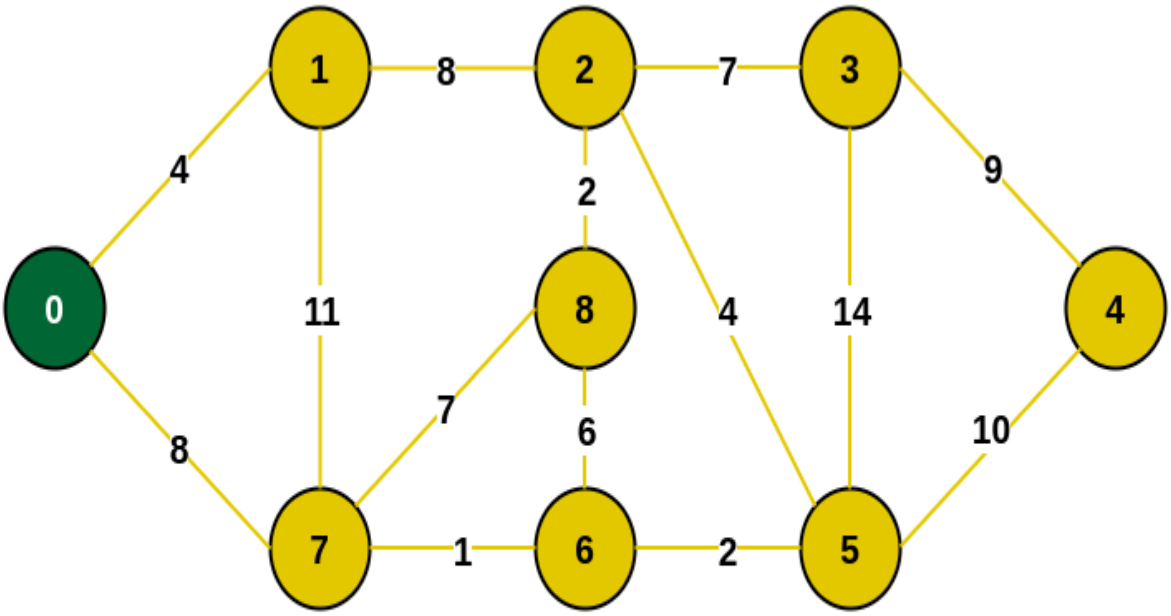
Consider the following graph as an example for which we need to find the Minimum Spanning Tree (MST).



Example of a Graph

Example of a graph

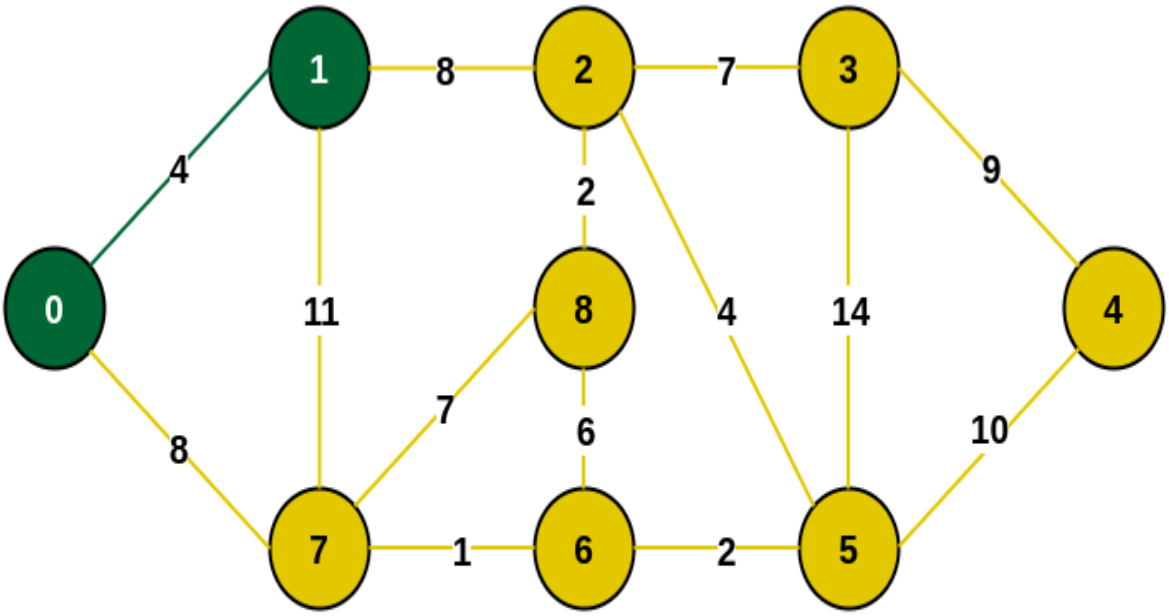
Step 1: *Firstly, we select an arbitrary vertex that acts as the starting vertex of the Minimum Spanning Tree. Here we have selected vertex **0** as the starting vertex.*



Select an arbitrary starting vertex. Here we have selected 0

0 is selected as starting vertex

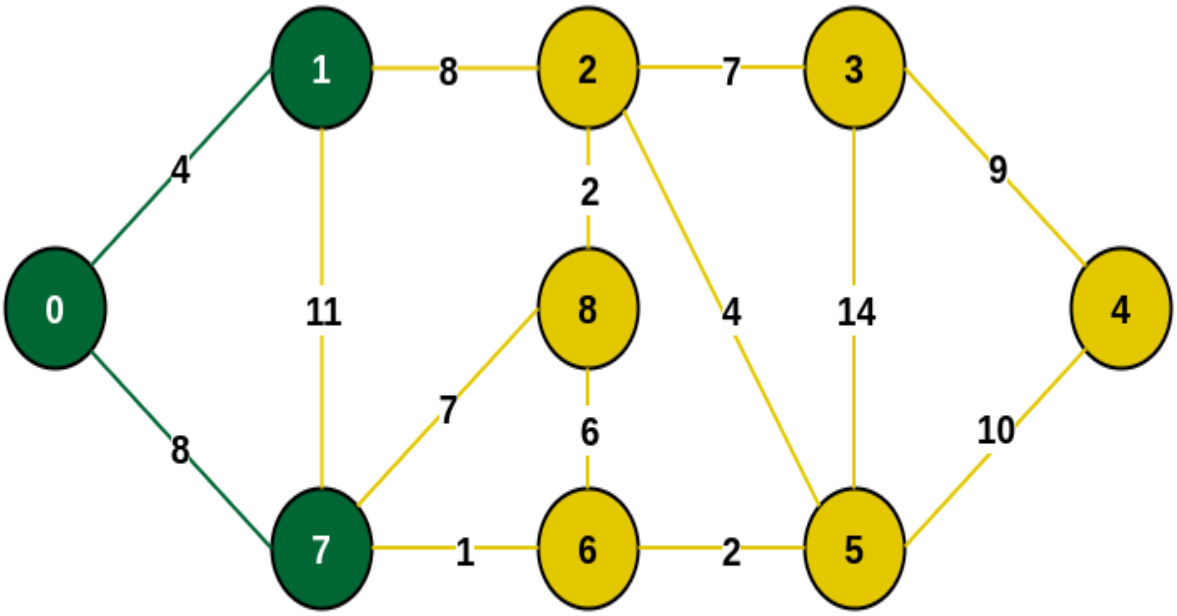
Step 2: *All the edges connecting the incomplete MST and other vertices are the edges {0, 1} and {0, 7}. Between these two the edge with minimum weight is {0, 1}. So include the edge and vertex 1 in the MST.*



Minimum weighted edge from MST to other vertices is 0-1 with weight 4

1 is added to the MST

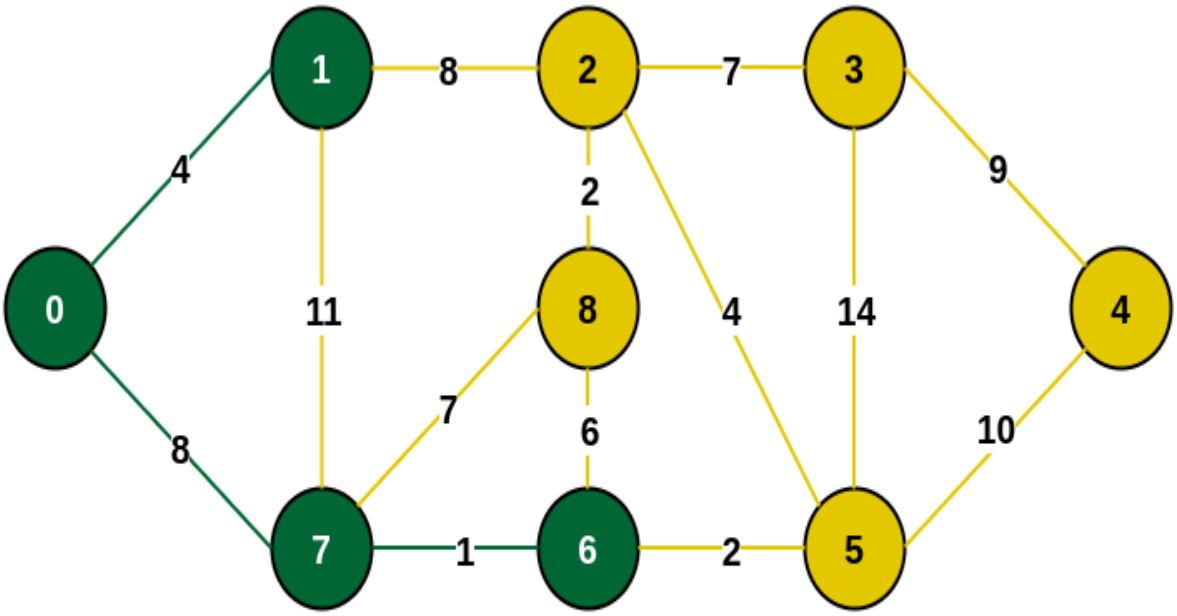
Step 3: *The edges connecting the incomplete MST to other vertices are {0, 7}, {1, 7} and {1, 2}. Among these edges the minimum weight is 8 which is of the edges {0, 7} and {1, 2}. Let us here include the edge {0, 7} and the vertex 7 in the MST. [We could have also included edge {1, 2} and vertex 2 in the MST].*



Minimum weighted edge from MST to other vertices is 0-7 with weight 8

7 is added in the MST

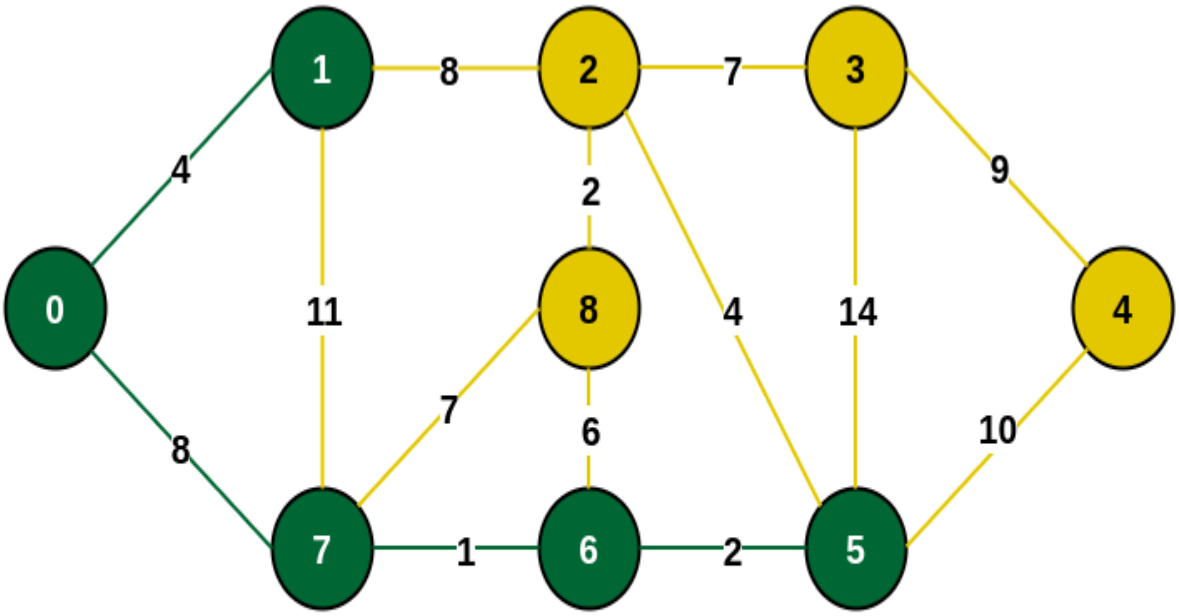
Step 4: *The edges that connect the incomplete MST with the fringe vertices are {1, 2}, {7, 6} and {7, 8}. Add the edge {7, 6} and the vertex 6 in the MST as it has the least weight (i.e., 1).*



Minimum weighted edge from MST to other vertices is 7-6 with weight 1

6 is added in the MST

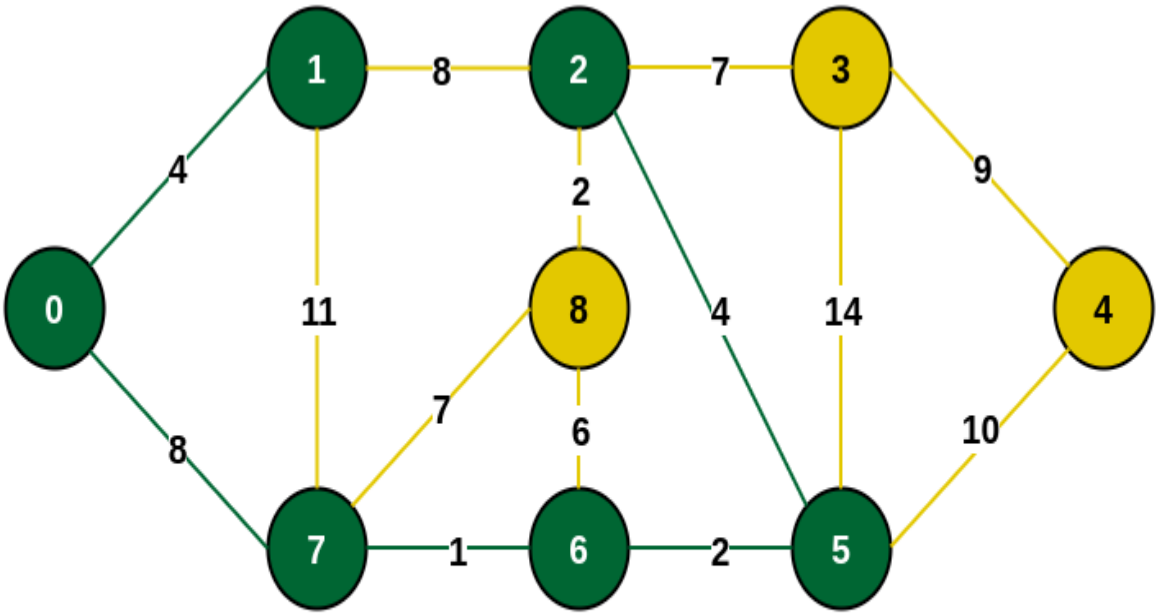
Step 5: *The connecting edges now are {7, 8}, {1, 2}, {6, 8} and {6, 5}. Include edge {6, 5} and vertex 5 in the MST as the edge has the minimum weight (i.e., 2) among them.*



Minimum weighted edge from MST to other vertices is 6-5 with weight 2

Include vertex 5 in the MST

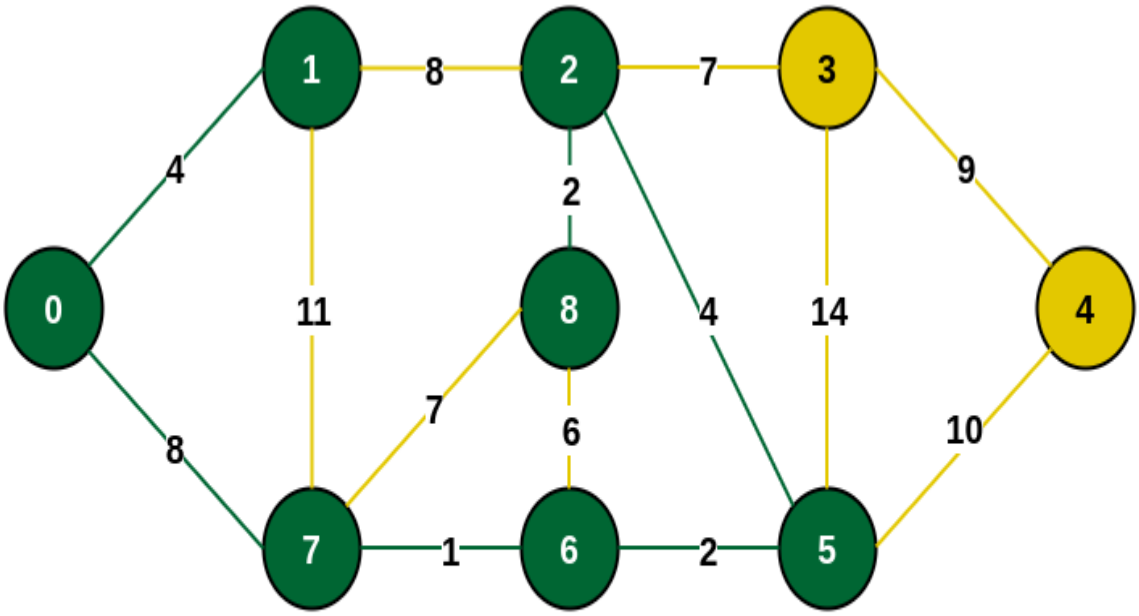
Step 6: *Among the current connecting edges, the edge {5, 2} has the minimum weight. So include that edge and the vertex 2 in the MST.*



Minimum weighted edge from MST to other vertices is 5-2 with weight 4

Include vertex 2 in the MST

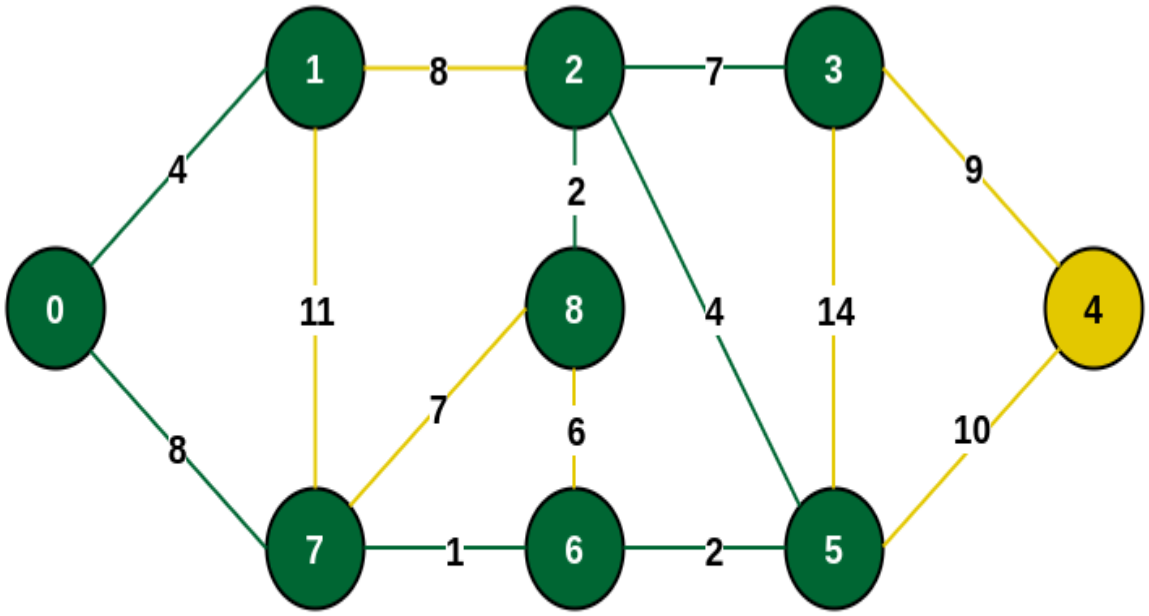
Step 7: *The connecting edges between the incomplete MST and the other edges are {2, 8}, {2, 3}, {5, 3} and {5, 4}. The edge with minimum weight is edge {2, 8} which has weight 2. So include this edge and the vertex 8 in the MST.*



Minimum weighted edge from MST to other vertices is 2-8 with weight 2

Add vertex 8 in the MST

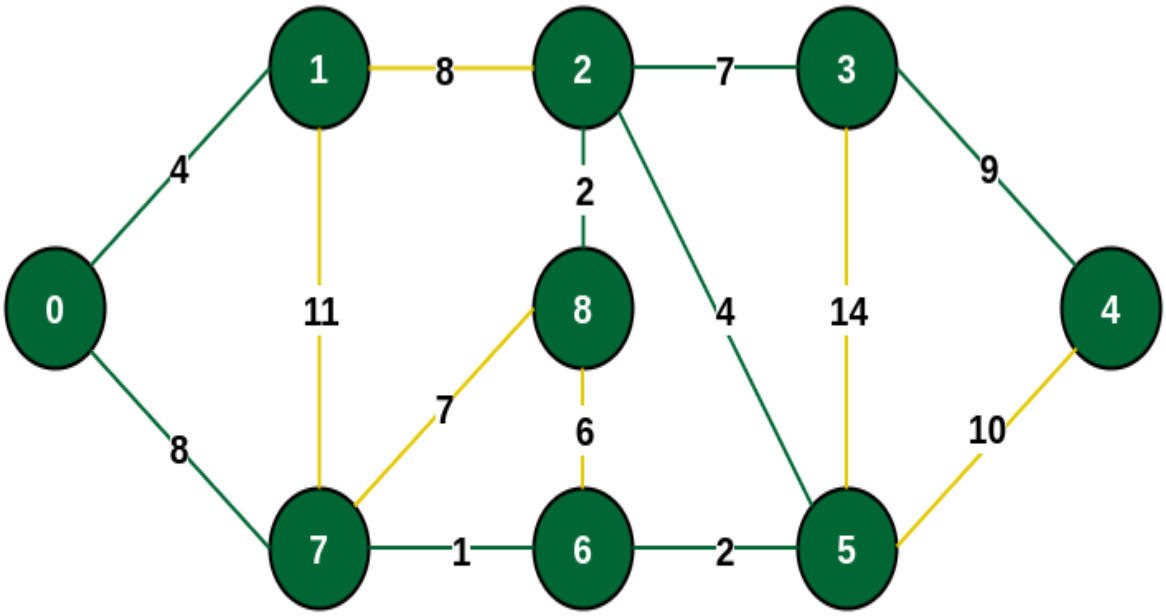
Step 8: *See here that the edges {7, 8} and {2, 3} both have same weight which are minimum. But 7 is already part of MST. So we will consider the edge {2, 3} and include that edge and vertex 3 in the MST.*



Minimum weighted edge from MST to other vertices is 2-3 with weight 7

Include vertex 3 in MST

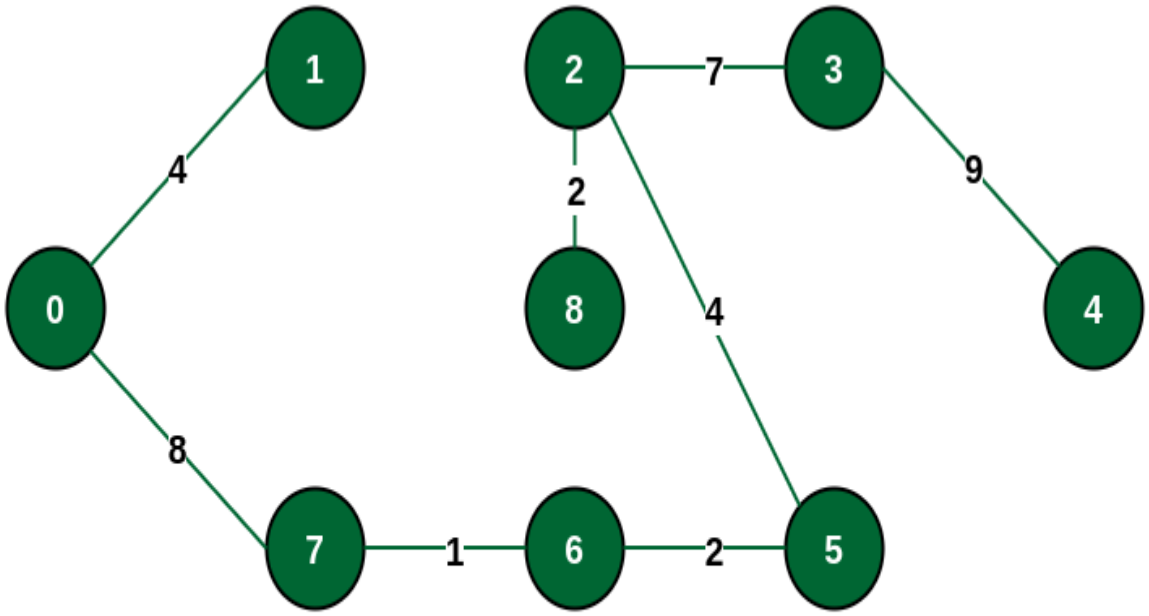
Step 9: *Only the vertex 4 remains to be included. The minimum weighted edge from the incomplete MST to 4 is {3, 4}.*



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

Include vertex 4 in the MST

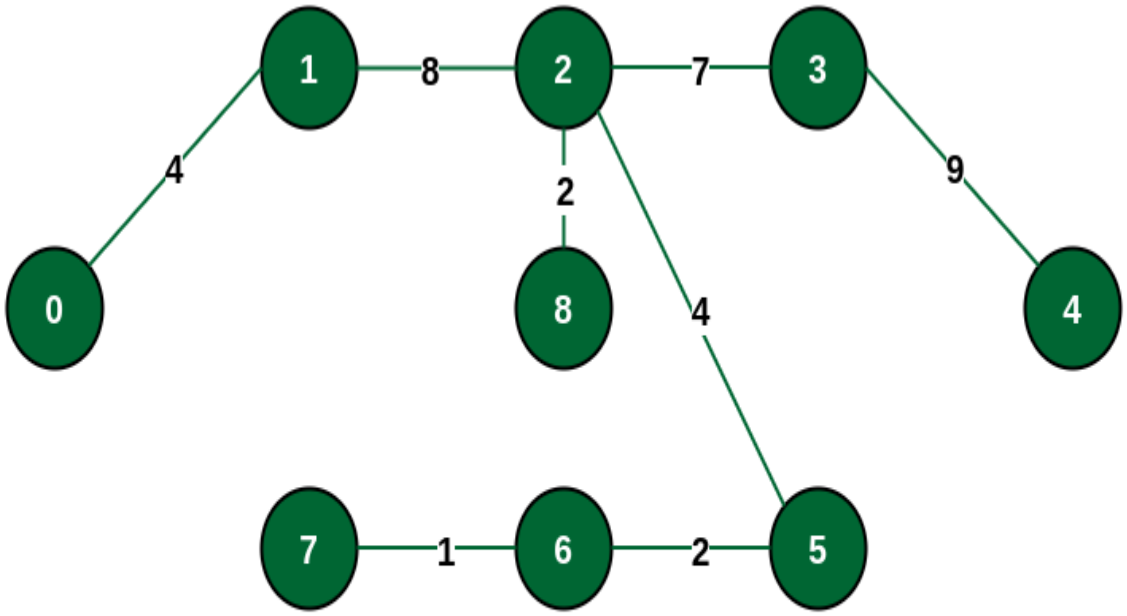
The final structure of the MST is as follows and the weight of the edges of the MST is (4 + 8 + 1 + 2 + 4 + 2 + 7 + 9) = 37.



The final structure of MST

The structure of the MST formed using the above method

Note: *If we had selected the edge {1, 2} in the third step then the MST would look like the following.*



Alternative MST structure

Structure of the alternate MST if we had selected edge {1, 2} in the MST

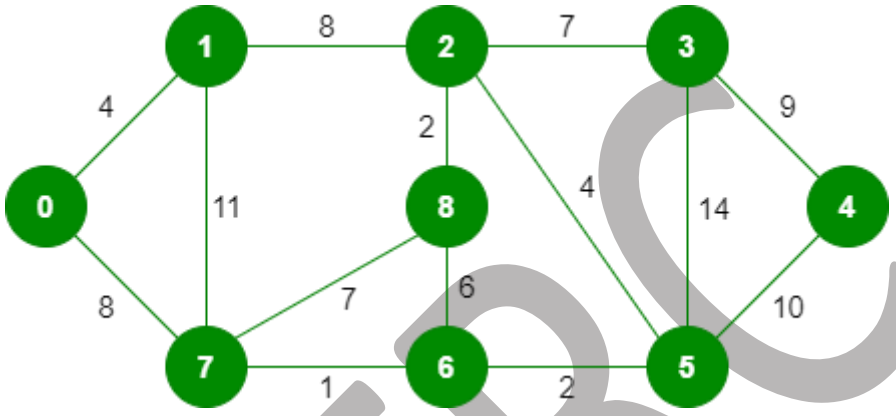
UNIT-5 (GRAPHS & HASHING)

MINIMUM SPANNING TREE : KRUSKAL'S

LECTURE NO:-37

Below is the illustration of the above approach:

Input Graph:



The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having (9 – 1) = 8 edges.

After sorting:

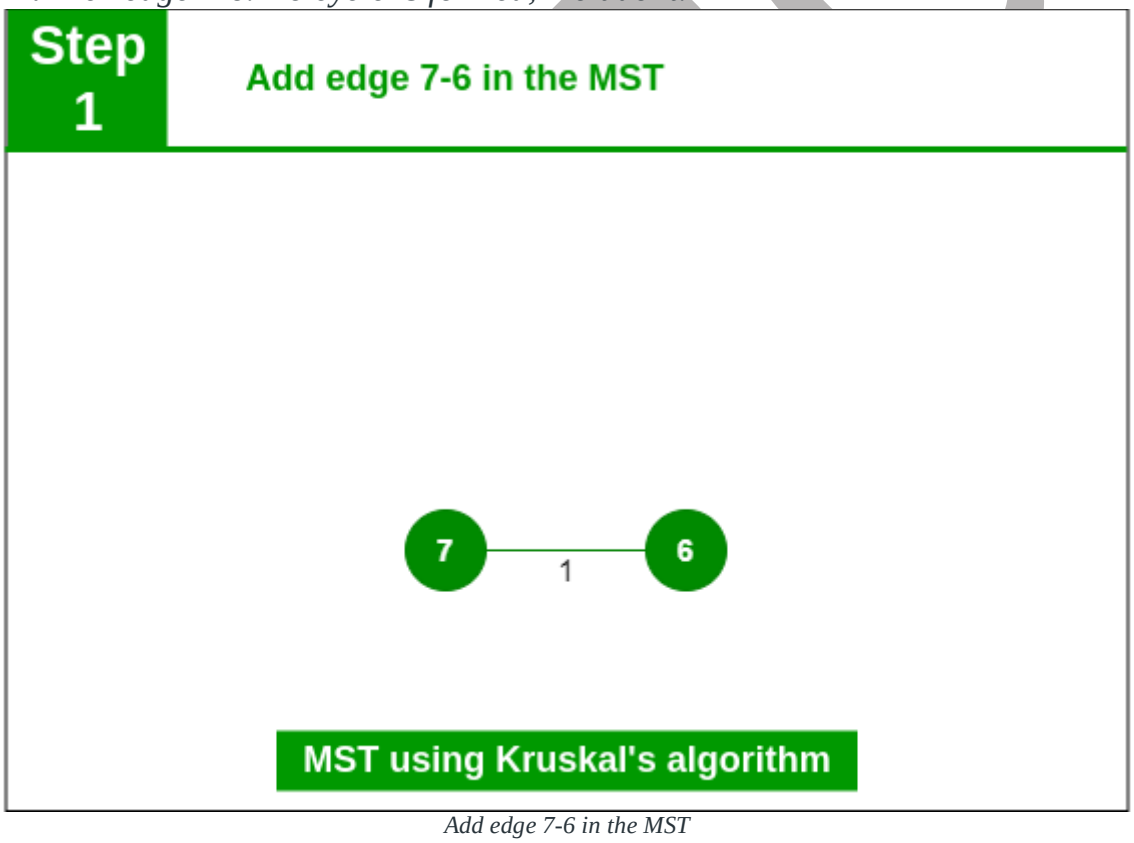
Weight	Source	Destination
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7

MITRC

Weight	Source	Destination
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

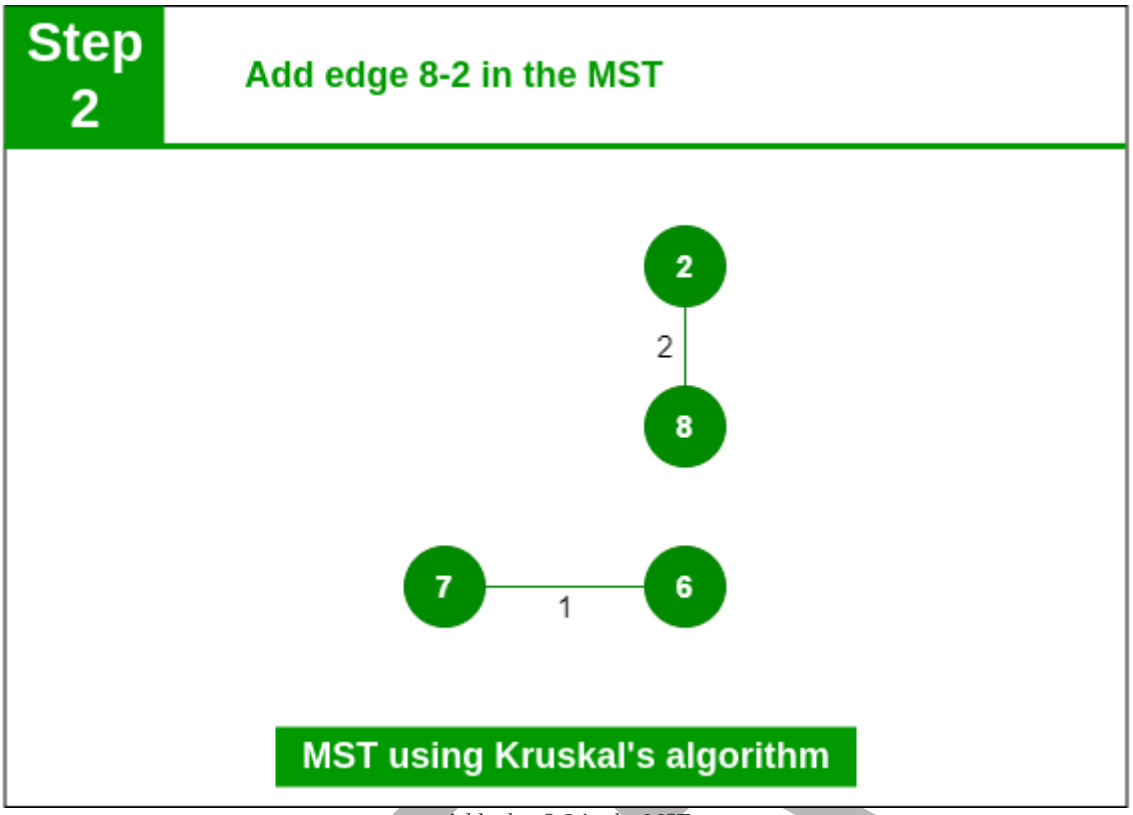
Now pick all edges one by one from the sorted list of edges

Step 1: *Pick edge 7-6. No cycle is formed, include it.*



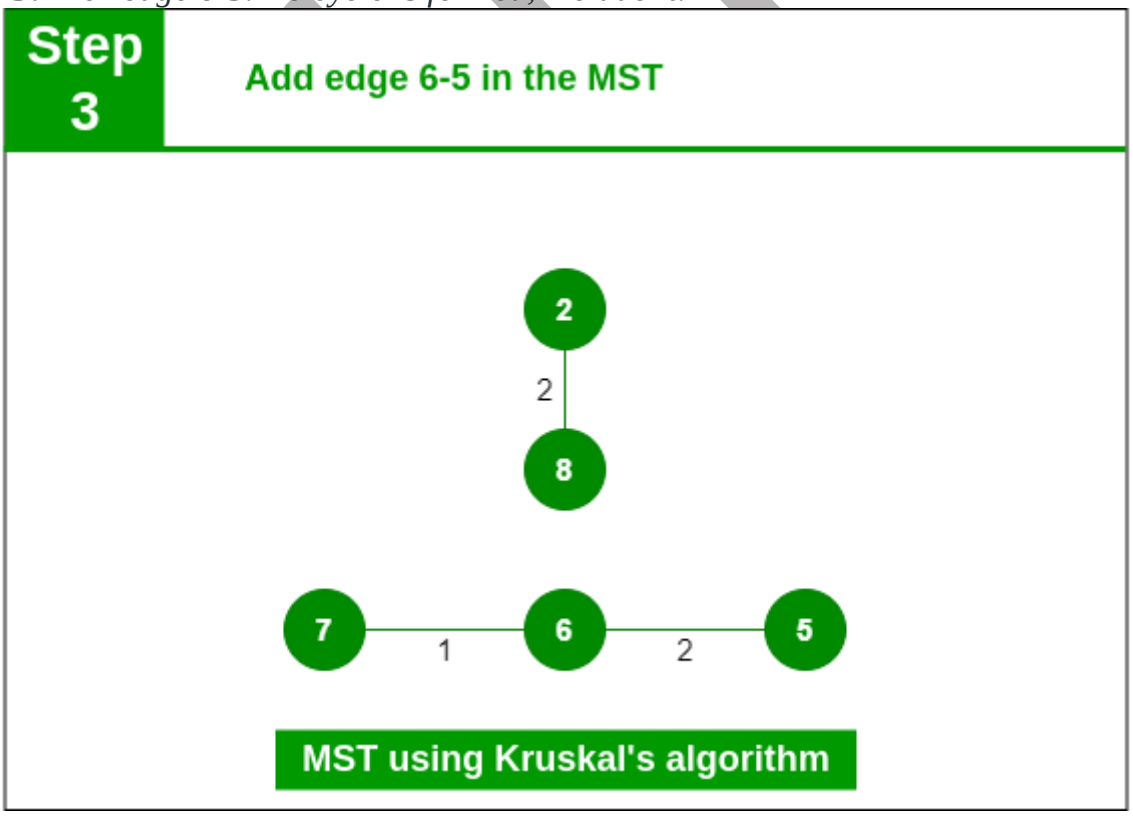
Add edge 7-6 in the MST

Step 2: *Pick edge 8-2. No cycle is formed, include it.*



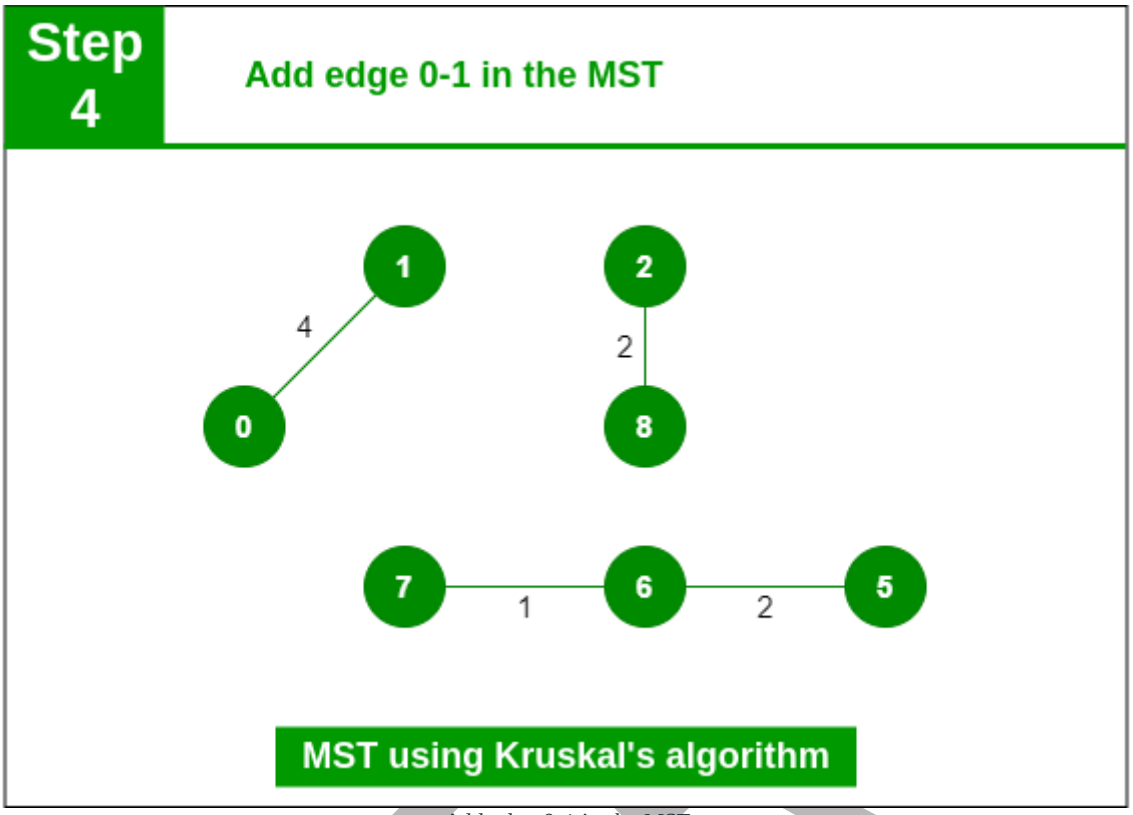
Add edge 8-2 in the MST

Step 3: *Pick edge 6-5. No cycle is formed, include it.*



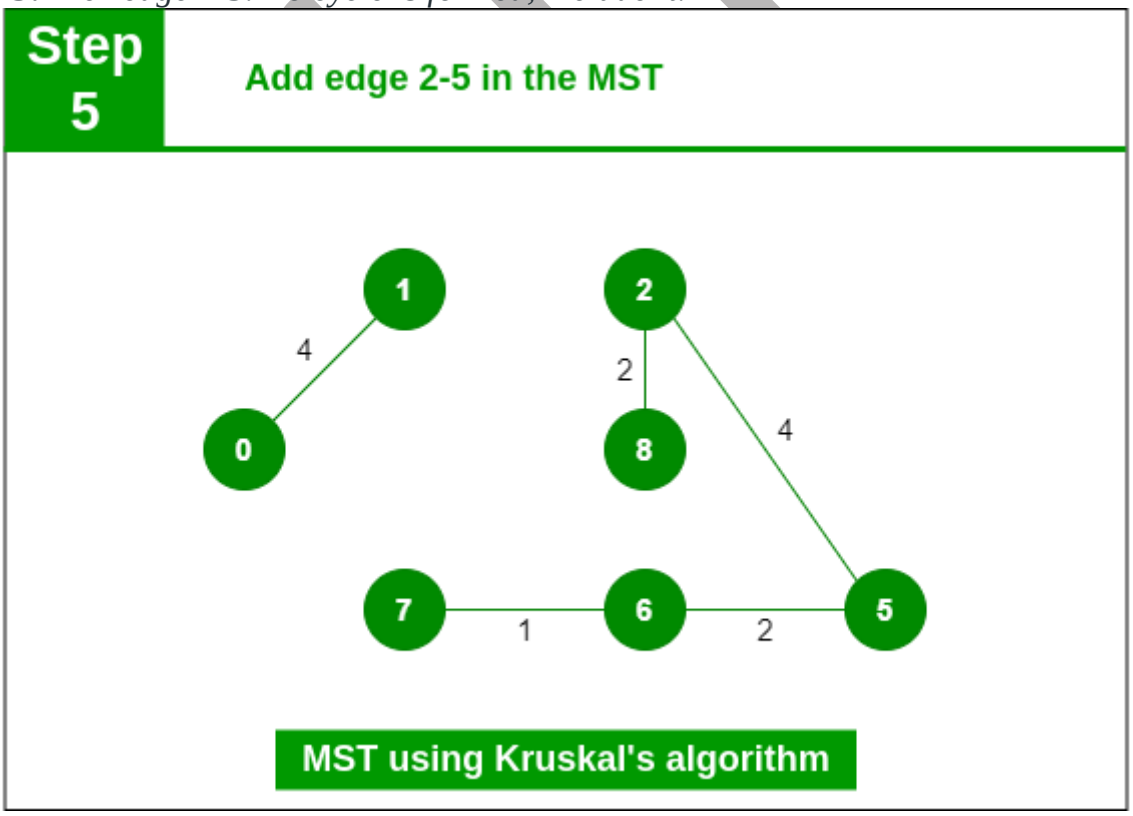
Add edge 6-5 in the MST

Step 4: *Pick edge 0-1. No cycle is formed, include it.*



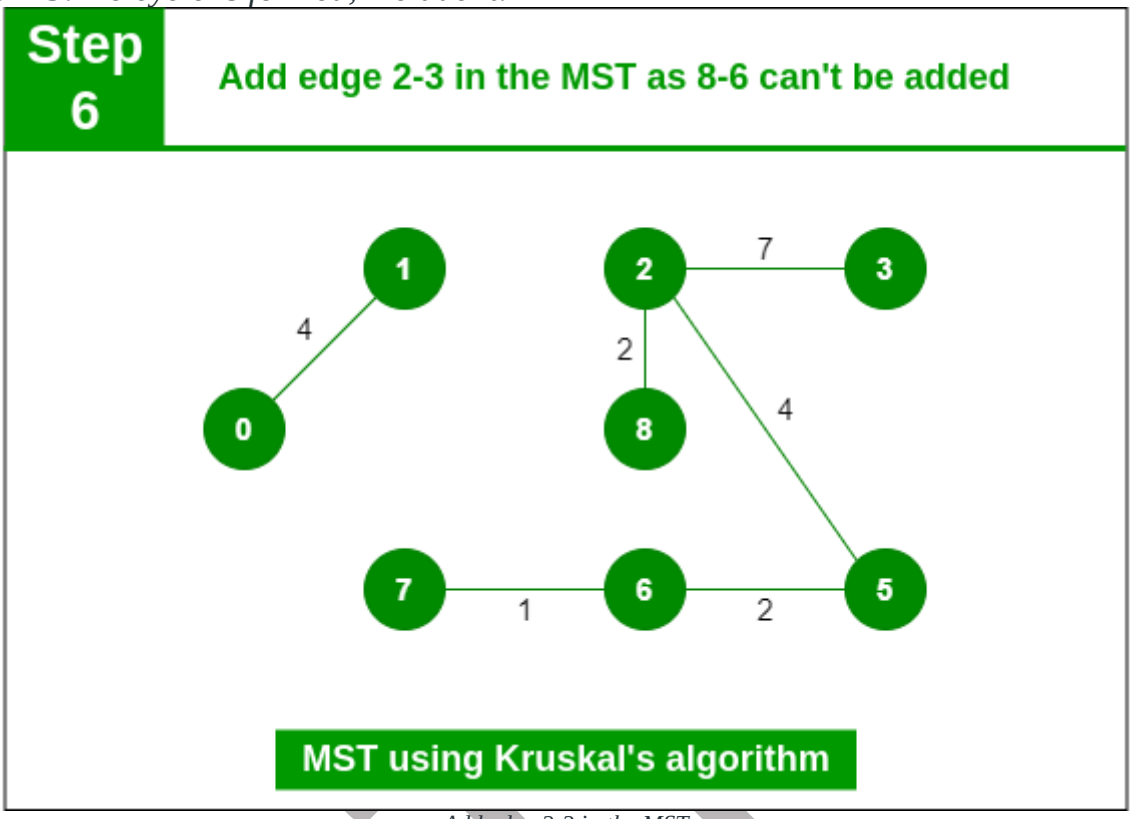
Add edge 0-1 in the MST

Step 5: *Pick edge 2-5. No cycle is formed, include it.*



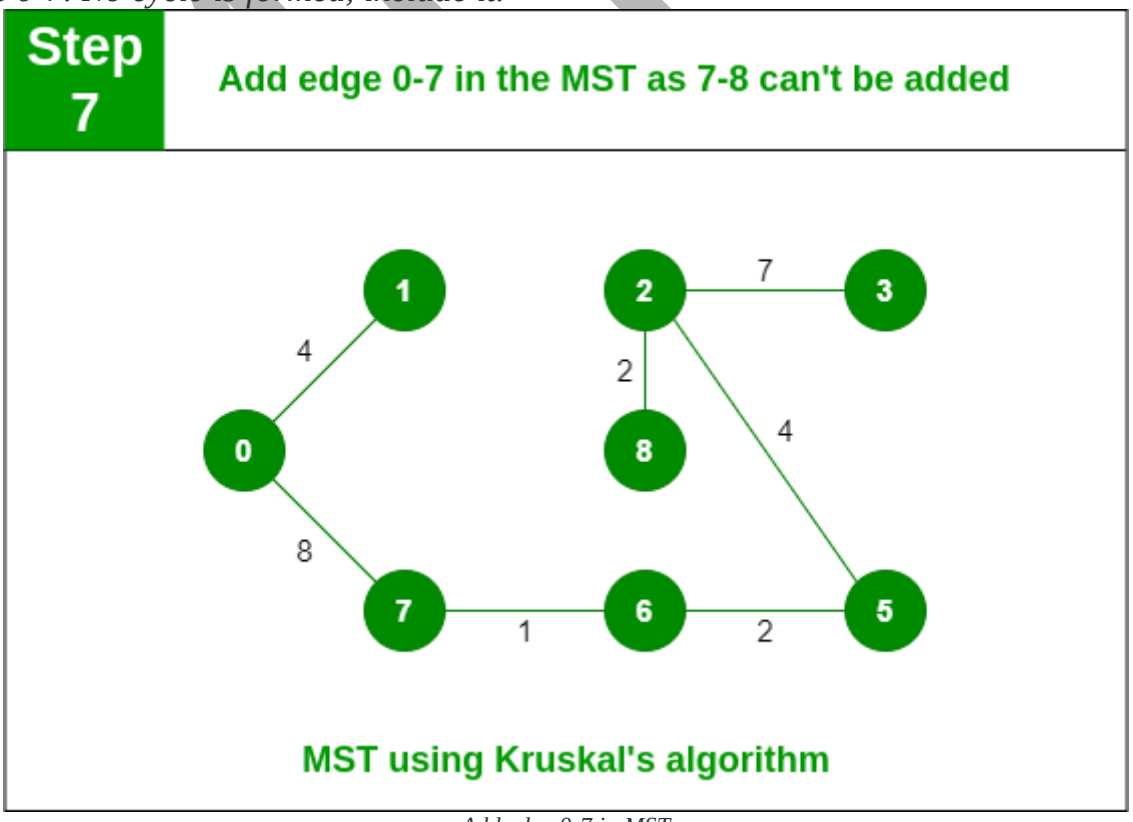
Add edge 2-5 in the MST

Step 6: *Pick edge 8-6. Since including this edge results in the cycle, discard it. Pick edge 2-3: No cycle is formed, include it.*



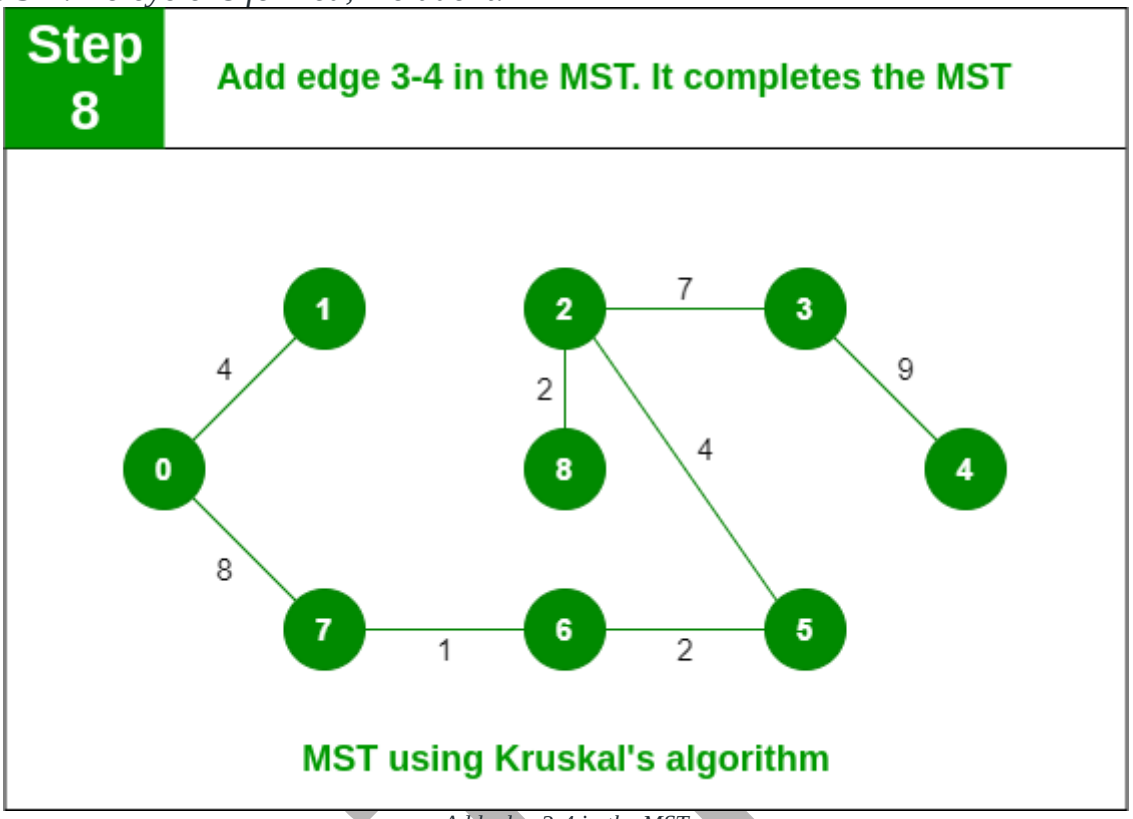
Add edge 2-3 in the MST

Step 7: *Pick edge 7-8. Since including this edge results in the cycle, discard it. Pick edge 0-7. No cycle is formed, include it.*



Add edge 0-7 in MST

Step 8: *Pick edge 1-2. Since including this edge results in the cycle, discard it. Pick edge 3-4. No cycle is formed, include it.*



Add edge 3-4 in the MST

Note: *Since the number of edges included in the MST equals to (V – 1), so the algorithm stops here*

UNIT-5 (GRAPHS & HASHING)

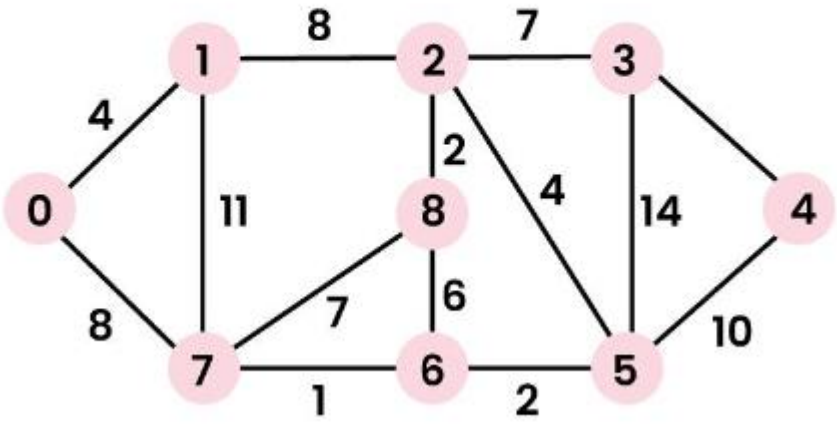
SINGLE SOURCE SHORTEST PATH: DIJKSTRA'S ALGORITHM

LECTURE NO:-38

The following is the step that we will follow to implement Dijkstra's Algorithm:

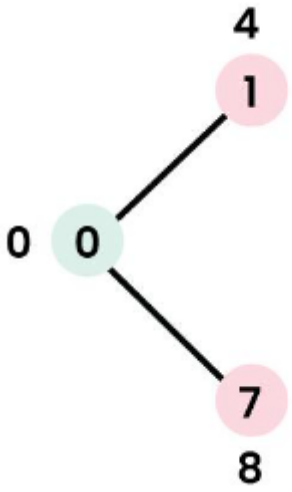
- Create a set **sptSet** (shortest path tree set) that keeps track of vertices included in the shortest path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.
- Assign a distance value to all vertices in the input graph. Initialize all distance values as **INFINITE**. Assign the distance value as 0 for the source vertex so that it is picked first.
- While **sptSet** doesn't include all vertices
    - Pick a vertex **u** that is not there in **sptSet** and has a minimum distance value.
    - Include u to **sptSet**.
    - Then update the distance value of all adjacent vertices of **u**.
        - To update the distance values, iterate through all adjacent vertices.
        - For every adjacent vertex **v,** if the sum of the distance value of **u** (from source) and weight of edge **u-v**, is less than the distance value of **v**, then update the distance value of **v**.

*Consider below graph and **src** = 0*



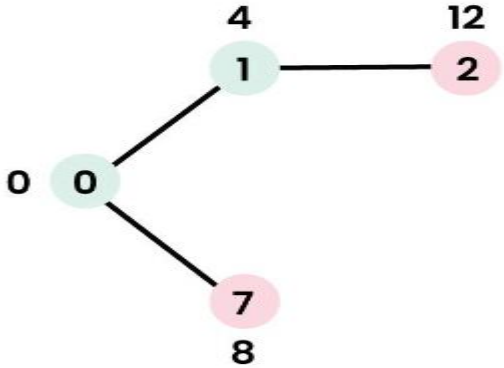
Step 1:

- *The set **sptSet** is initially empty and distances assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF} where **INF** indicates infinite.*
- *Now pick the vertex with a minimum distance value. The vertex 0 is picked, include it in **sptSet**. So **sptSet** becomes {0}. After including 0 to **sptSet**, update distance values of its adjacent vertices.*
- *Adjacent vertices of 0 are 1 and 7. The distance values of 1 and 7 are updated as 4 and 8.*



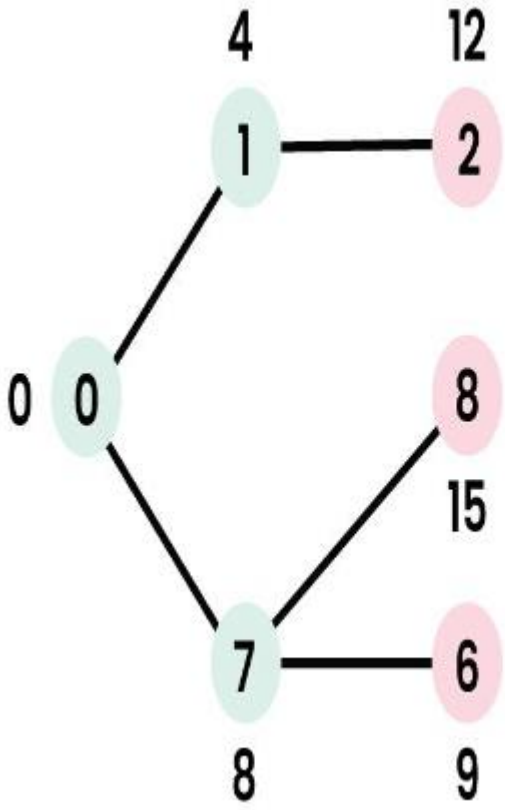
Step 2:

- *Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSET**). The vertex 1 is picked and added to **sptSet**.*
- *So **sptSet** now becomes {0, 1}. Update the distance values of adjacent vertices of 1.*
- *The distance value of vertex 2 becomes **12**.*



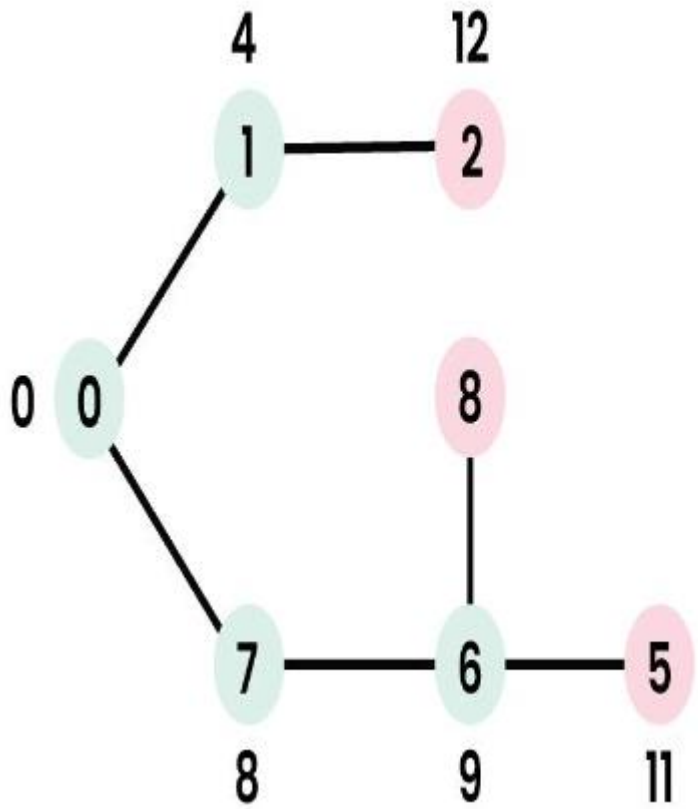
Step 3:

- *Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSET**). Vertex 7 is picked. So **sptSet** now becomes **{0, 1, 7}**.*
- *Update the distance values of adjacent vertices of 7. The distance value of vertex 6 and 8 becomes finite (**15 and 9** respectively).*

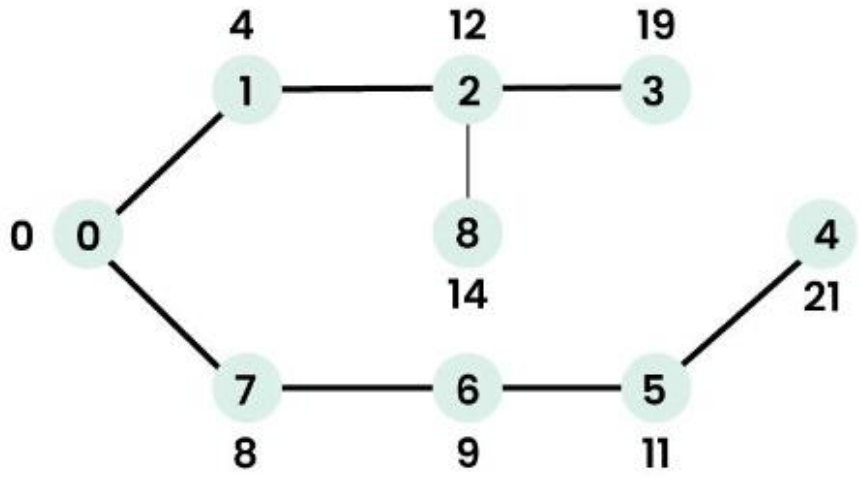


Step 4:

- *Pick the vertex with minimum distance value and not already included in **SPT** (not in **sptSET**). Vertex 6 is picked. So **sptSet** now becomes **{0, 1, 7, 6}**.*
- *Update the distance values of adjacent vertices of 6. The distance value of vertex 5 and 8 are updated.*



We repeat the above steps until ***sptSet*** *includes all vertices of the given graph. Finally, we get the following* ***Shortest Path Tree (SPT).***



Applications of Dijkstra's Algorithm:

- **Google maps** uses Dijkstra algorithm to show shortest distance between source and destination.
- In **computer networking**, Dijkstra's algorithm forms the basis for various routing protocols, such as OSPF (Open Shortest Path First) and IS-IS (Intermediate System to Intermediate System).
- Transportation and traffic management systems use Dijkstra's algorithm to optimize traffic flow, minimize congestion, and plan the most efficient routes for vehicles.
- Airlines use Dijkstra's algorithm to plan flight paths that minimize fuel consumption, reduce travel time.
- Dijkstra's algorithm is applied in electronic design automation for routing connections on integrated circuits and very-large-scale integration (VLSI) chips.

Advantages

1. One primary advantage of using Dijkstra's Algorithm is that it has an almost linear time and space complexity.
2. We can use this algorithm to calculate the shortest path from a single vertex to all other vertices and a single source vertex to a single destination vertex by stopping the algorithm once we get the shortest distance for the destination vertex.
3. This algorithm only works for directed weighted graphs, and all the edges of this graph should be non-negative.

Disadvantages

1. Dijkstra's Algorithm performs a concealed exploration that utilizes a lot of time during the process.
2. This algorithm is impotent to handle negative edges.
3. Since this algorithm heads to the acyclic graph, it cannot calculate the exact shortest path.
4. It also requires maintenance to keep a record of vertices that have been visited.

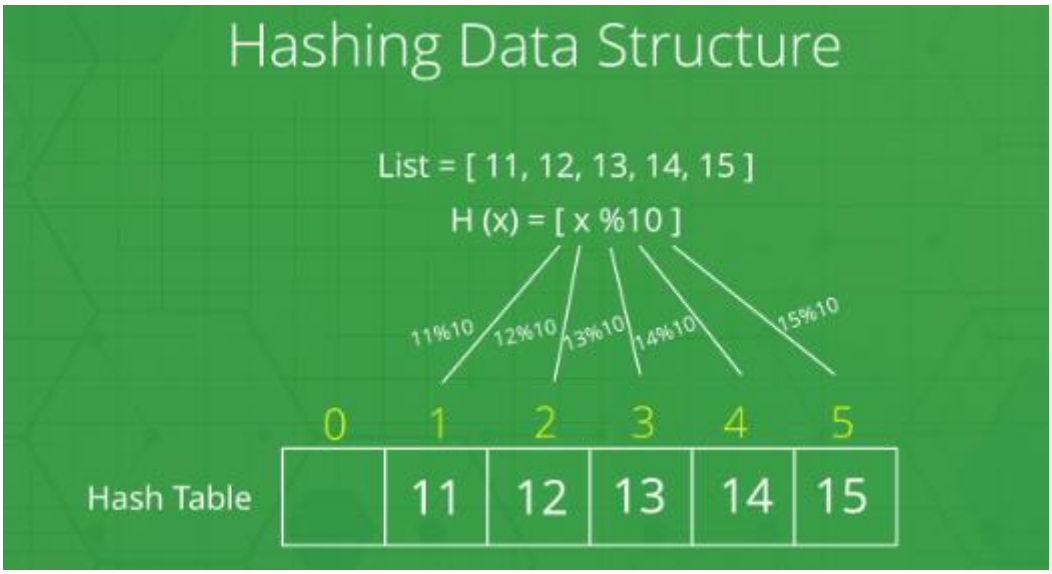
UNIT-5 (GRAPHS & HASHING)

HASHING: INTRODUCTION, OPEN HASHING-SEPARATE CHAINING

LECTURE NO:-39

Hashing is a technique or process of mapping keys, and values into the hash table by using a hash function. It is done for faster access to elements. The efficiency of mapping depends on the efficiency of the hash function used.

Let a hash function H(x) maps the value **x** at the index **x%10** in an Array. For example if the list of values is [11,12,13,14,15] it will be stored at positions {1,2,3,4,5} in the array or Hash table respectively.



Since a hash function gets us a small number for a key which is a big integer or string, there is a possibility that two keys result in the same value. The situation where a newly inserted key maps to an already occupied slot in the hash table is called collision and must be handled using some collision handling technique.

There are mainly two methods to handle collision:

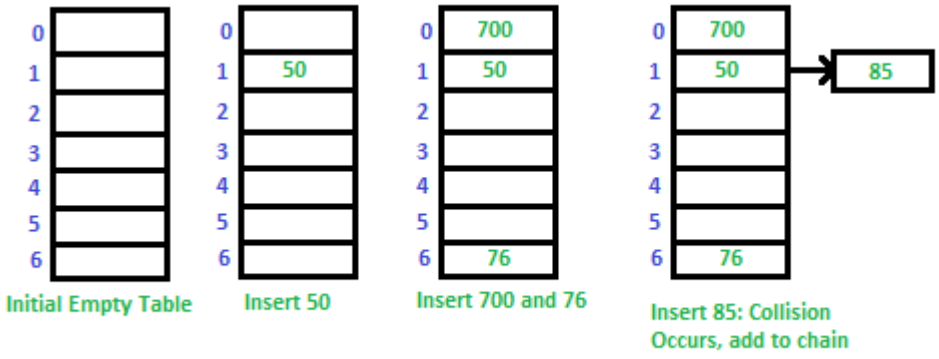
- Open Hashing (or) Separate Chaining
- Closed Hashing (or) Open Addressing

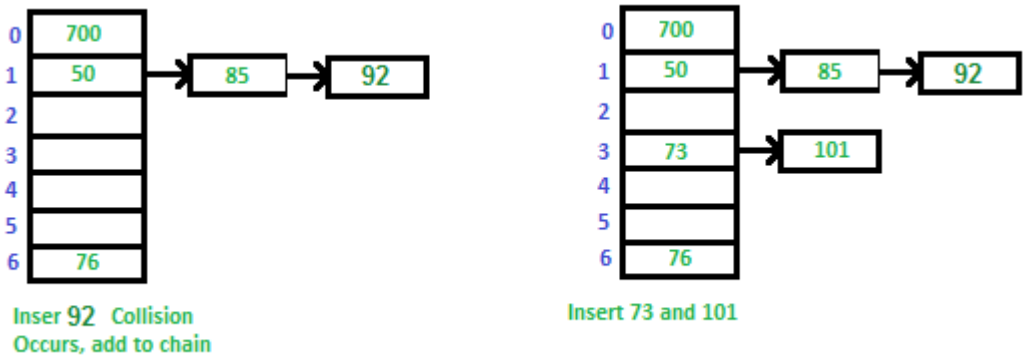
Open Hashing (or) Separate Chaining

The idea behind separate chaining is to implement the array as a linked list called a chain. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions.

The **linked list** data structure is used to implement this technique. So what happens is, when multiple elements are hashed into the same slot index, then these elements are inserted into a singly-linked list which is known as a chain.

Example: Let us consider a simple hash function as "**key mod 7**" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101





Advantages:

- Simple to implement.
- Hash table never fills up, we can always add more elements to the chain.
- Less sensitive to the hash function or load factors.
- It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.

Disadvantages:

- The cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.
- Wastage of Space (Some Parts of the hash table are never used)
- If the chain becomes long, then search time can become O(n) in the worst case
- Uses extra space for links

MITRC

UNIT-5 (GRAPHS & HASHING)

CLOSED HASHING - LINEAR PROBING, QUADRATIC PROBING

LECTURE NO:-40

Closed Hashing (or) Open Addressing

Like separate chaining, open addressing is a method for handling collisions. In Open Addressing, all elements are stored in the **hash table** itself. So at any point, the size of the table must be greater than or equal to the total number of keys (Note that we can increase table size by copying old data if needed). This approach is also known as closed hashing. This entire procedure is based upon probing.

- **Insert(k):** Keep probing until an empty slot is found. Once an empty slot is found, insert k.
- **Search(k):** Keep probing until the slot's key doesn't become equal to k or an empty slot is reached.
- **Delete(k)**: **Delete operation is interesting**. If we simply delete a key, then the search may fail. So slots of deleted keys are marked specially as "deleted".
  The insert can insert an item in a deleted slot, but the search doesn't stop at a deleted slot.

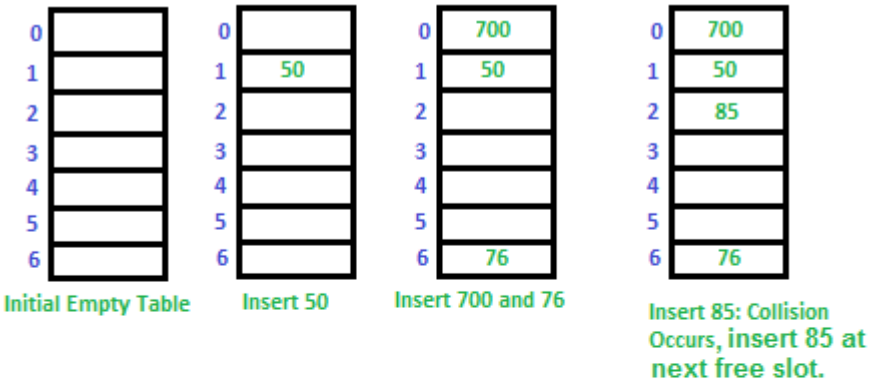
1. Linear Probing:

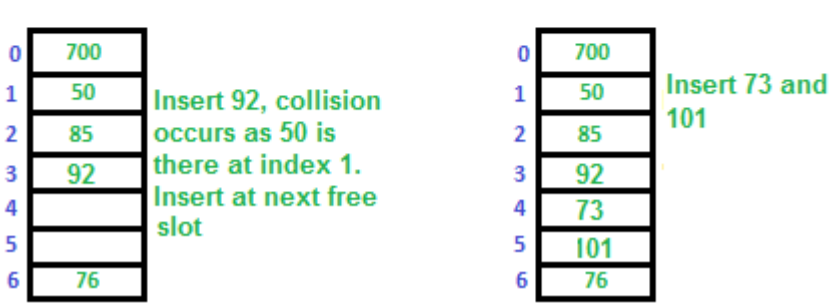
In linear probing, the hash table is searched sequentially that starts from the original location of the hash. If in case the location that we get is already occupied, then we check for the next location.

The function used for rehashing is as follows: rehash(key) = (n+1)%table-size.

Let us consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101,

which means hash(key)= key% S, here S=size of the table =7,indexed from 0 to 6.We can define the hash function as per our choice if we want to create a hash table,although it is fixed internally with a pre-defined formula.





Applications of linear probing:

Linear probing is a collision handling technique used in hashing, where the algorithm looks for the next available slot in the hash table to store the collided key. Some of the applications of linear probing include:

- **Symbol tables**: Linear probing is commonly used in symbol tables, which are used in compilers and interpreters to store variables and their associated values. Since symbol tables can grow dynamically, linear probing can be used to handle collisions and ensure that variables are stored efficiently.
- **Caching**: Linear probing can be used in caching systems to store frequently accessed data in memory. When a cache miss occurs, the data can be loaded into the cache using linear probing, and when a collision occurs, the next available slot in the cache can be used to store the data.
- **Databases**: Linear probing can be used in databases to store records and their associated keys. When a collision occurs, linear probing can be used to find the next available slot to store the record.
- **Compiler design**: Linear probing can be used in compiler design to implement symbol tables, error recovery mechanisms, and syntax analysis.

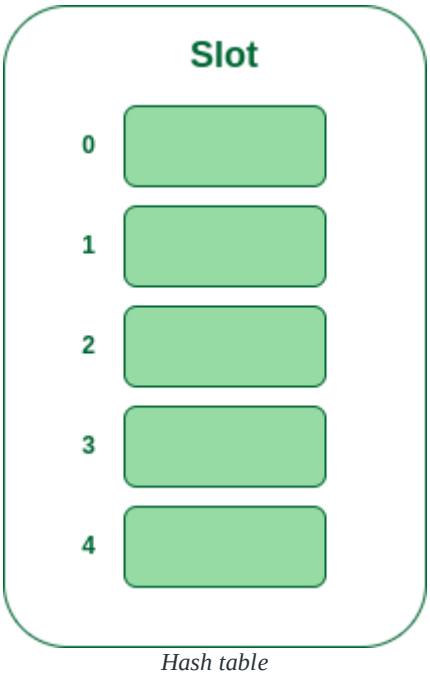
- **Spell checking:** Linear probing can be used in spell-checking software to store the dictionary of words and their associated frequency counts. When a collision occurs, linear probing can be used to store the word in the next available slot.

Challenges in Linear Probing :

- **Primary Clustering:** One of the problems with linear probing is Primary clustering, many consecutive elements form groups and it starts taking time to find a free slot or to search for an element.
- **Secondary Clustering*:*** Secondary clustering is less severe, two records only have the same collision chain (Probe Sequence) if their initial position is the same.

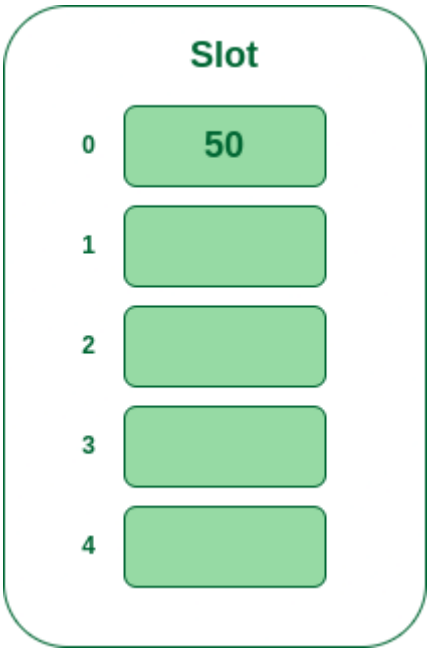
Example: Let us consider a simple hash function as “key mod 5” and a sequence of keys that are to be inserted are 50, 70, 76, 93.

- **Step1:** First draw the empty hash table which will have a possible range of hash values from 0 to 4 according to the hash function provided.



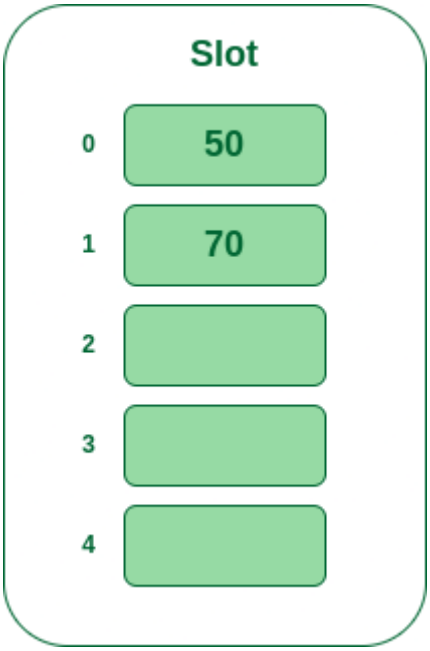
Hash table

- **Step 2:** Now insert all the keys in the hash table one by one. The first key is 50. It will map to slot number 0 because 50%5=0. So insert it into slot number 0.



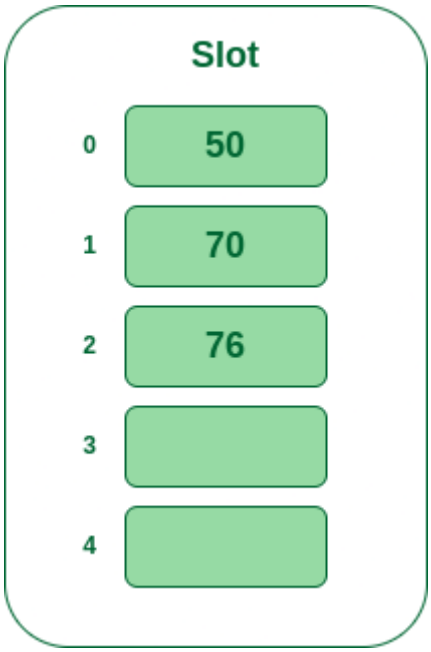
Insert 50 into hash table

- **Step 3:** The next key is 70. It will map to slot number 0 because 70%5=0 but 50 is already at slot number 0 so, search for the next empty slot and insert it.



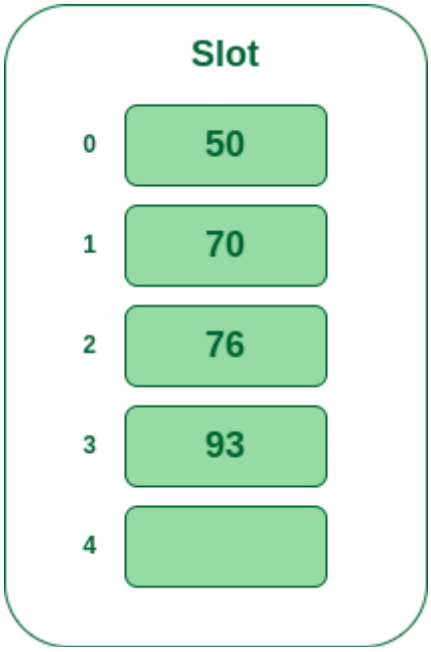
Insert 70 into hash table

- **Step 4:** The next key is 76. It will map to slot number 1 because 76%5=1 but 70 is already at slot number 1 so, search for the next empty slot and insert it.



Insert 76 into hash table

- **Step 5:** The next key is 93 It will map to slot number 3 because 93%5=3, So insert it into slot number 3.



Insert 93 into hash table

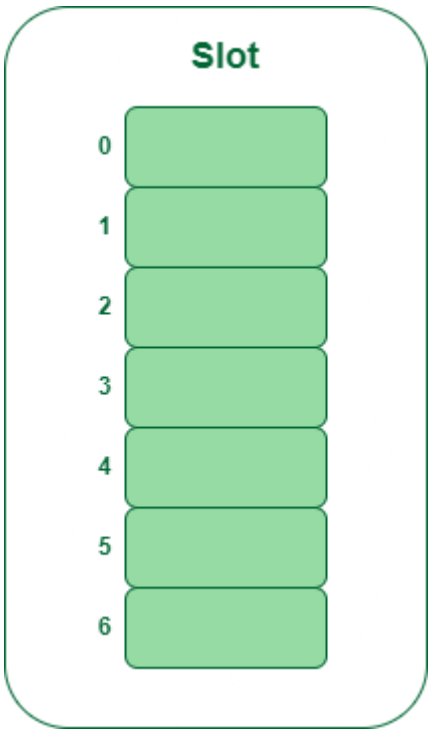
2. Quadratic Probing

If you observe carefully, then you will understand that the interval between probes will increase proportionally to the hash value. Quadratic probing is a method with the help of which we can solve the problem of clustering that was discussed above. This method is also known as the **mid-square** method. In this method, we look for the **i^2'th** slot in

the **i**th iteration. We always start from the original hash location. If only the location is occupied then we check the other slots.

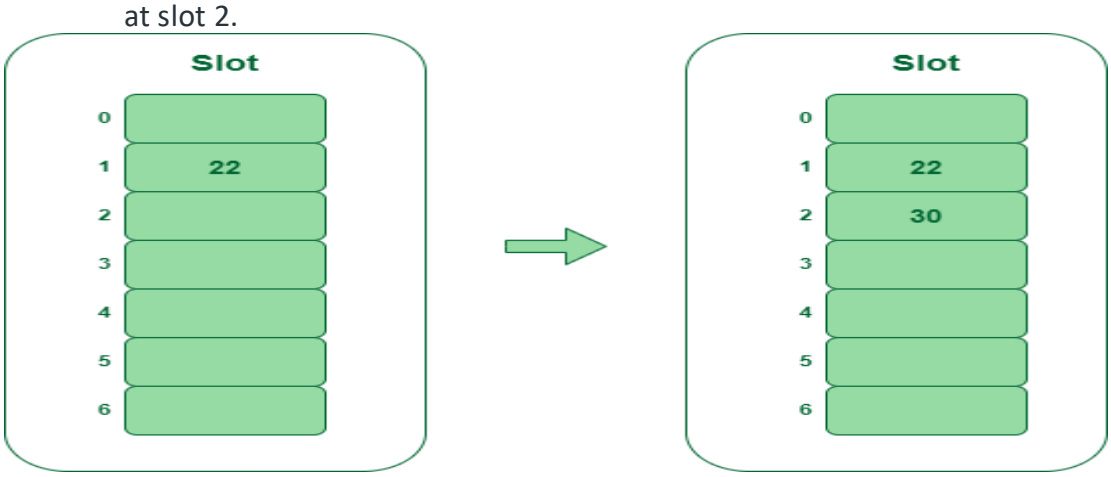
Example: Let us consider table Size = 7, hash function as Hash(x) = x % 7 and collision resolution strategy to be f(i) = i2 . Insert = 22, 30, and 50.

- Step 1: Create a table of size 7.



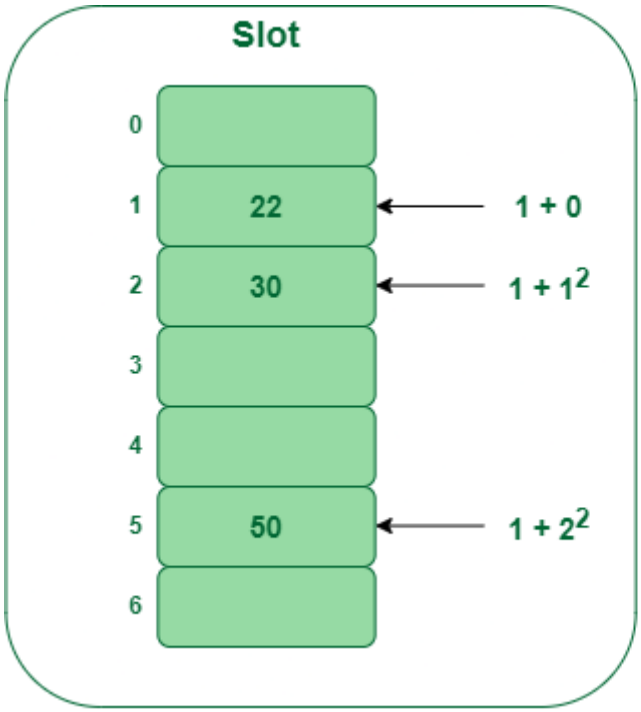
Hash table

- **Step 2** – Insert 22 and 30
  - Hash(22) = 22 % 7 = 1, Since the cell at index 1 is empty, we can easily insert 22 at slot 1.
  - Hash(30) = 30 % 7 = 2, Since the cell at index 2 is empty, we can easily insert 30 at slot 2.



Insert keys 22 and 30 in the hash table

- **Step 3:** Inserting 50
    - Hash(50) = 50 % 7 = 1
    - In our hash table slot 1 is already occupied. So, we will search for slot $1+1^2$, i.e. 1+1 = 2,
    - Again slot 2 is found occupied, so we will search for cell $1+2^2$, i.e.1+4 = 5,
    - Now, cell 5 is not occupied so we will place 50 in slot 5.



Insert key 50 in the hash table

UNIT-5 (GRAPHS & HASHING)

DOUBLE HASHING, RANDOM PROBING, REHASHING

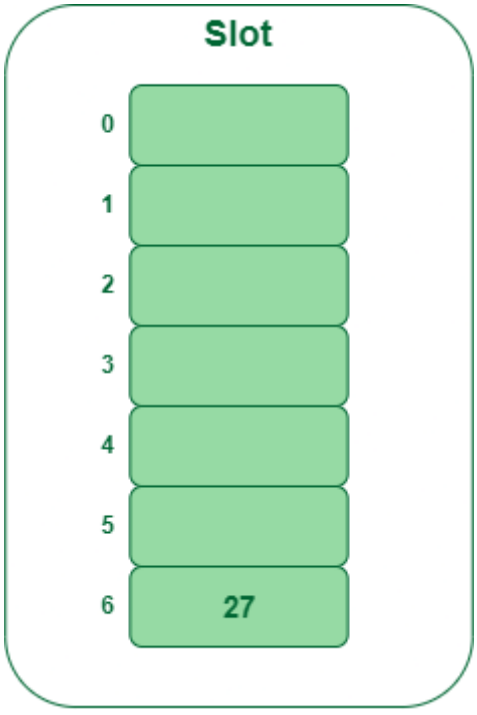
LECTURE NO:-41

Double Hashing

The intervals that lie between probes are computed by another hash function. Double hashing is a technique that reduces clustering in an optimized way. In this technique, the increments for the probing sequence are computed by using another hash function. We use another hash function hash2(x) and look for the i*hash2(x) slot in the **i**th rotation.

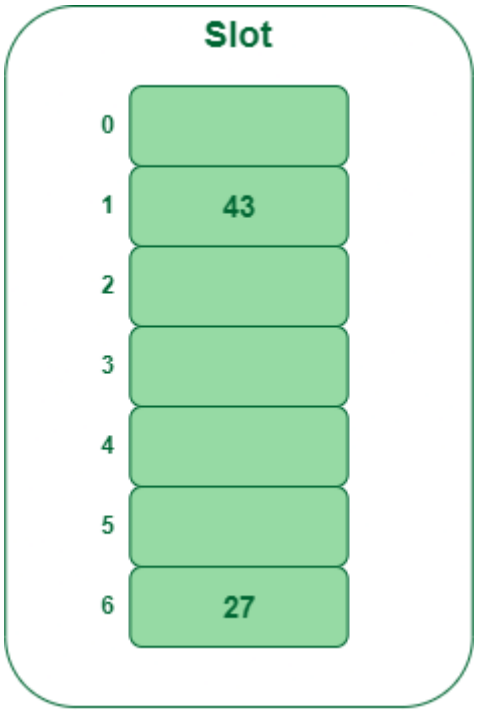
Example: Insert the keys 27, 43, 692, 72 into the Hash Table of size 7. where first hash-function is **h1(k) = k mod 7** and second hash-function is **h2(k) = 1 + (k mod 5)**

- **Step 1:** Insert 27
  - 27 % 7 = 6, location 6 is empty so insert 27 into 6 slot.



Insert key 27 in the hash table

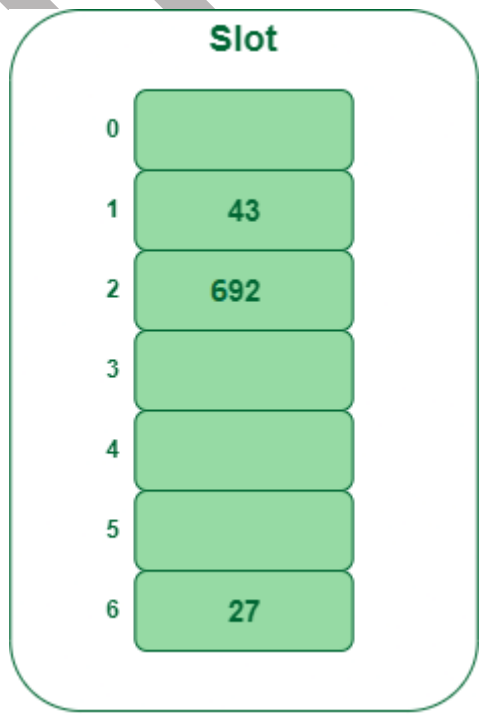
- **Step 2:** Insert 43
  - 43 % 7 = 1, location 1 is empty so insert 43 into 1 slot.



Insert key 43 in the hash table

- **Step 3:** Insert 692
    - 692 % 7 = 6, but location 6 is already being occupied and this is a collision
    - So we need to resolve this collision using double hashing.

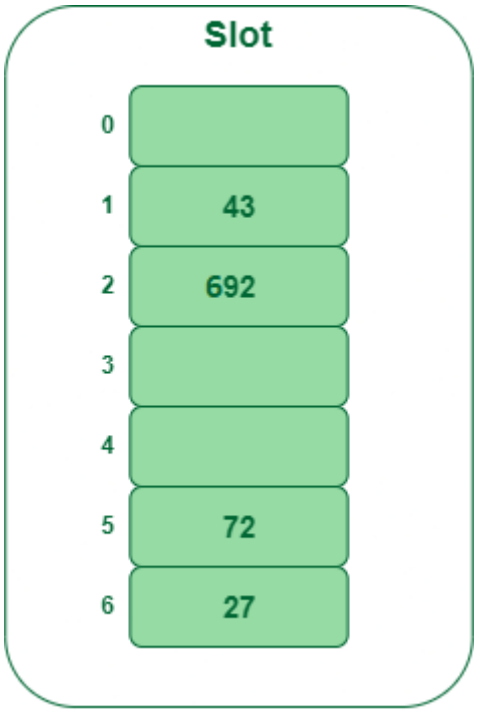
*h_{new} = [h1(692) + i * (h2(692)] % 7*
*= [6 + 1 * (1 + 692 % 5)] % 7*
= 9% 7
= 2
Now, as 2 is an empty slot,
so we can insert 692 into 2nd slot.



Insert key 692 in the hash table

- **Step 4:** Insert 72
    - 72 % 7 = 2, but location 2 is already being occupied and this is a collision.
    - So we need to resolve this collision using double hashing.

*h_{new} = [h1(72) + i * (h2(72)] % 7*
*= [2 + 1 * (1 + 72 % 5)] % 7*
= 5 % 7
= 5,
Now, as 5 is an empty slot,
so we can insert 72 into 5th slot.



Insert key 72 in the hash table

Comparison of the above three:

- In **linear probing**, the algorithm simply looks for the next available slot in the hash table and places the collided key there. If that slot is also occupied, the algorithm continues searching for the next available slot until an empty slot is found. This process is repeated until all collided keys have been stored. Linear probing has the best cache performance but suffers from clustering. One more advantage of Linear probing is easy to compute.
- In **quadratic probing**, the algorithm searches for slots in a more spaced-out manner. When a collision occurs, the algorithm looks for the next slot using an equation that involves the original hash value and a quadratic function. If that slot is also occupied, the algorithm increments the value of the quadratic function and tries again. This process is

repeated until an empty slot is found. Quadratic probing lies between the two in terms of cache performance and clustering.

- In **double hashing**, the algorithm uses a second hash function to determine the next slot to check when a collision occurs. The algorithm calculates a hash value using the original hash function, then uses the second hash function to calculate an offset. The algorithm then checks the slot that is the sum of the original hash value and the offset. If that slot is occupied, the algorithm increments the offset and tries again. This process is repeated until an empty slot is found. Double hashing has poor cache performance but no clustering. Double hashing requires more computation time as two hash functions need to be computed.

MITRC

S.No.	Separate Chaining	Open Addressing
1.	Chaining is Simpler to implement.	Open Addressing requires more computation.
2.	In chaining, Hash table never fills up, we can always add more elements to chain.	In open addressing, table may become full.
3.	Chaining is Less sensitive to the hash function or load factors.	Open addressing requires extra care to avoid clustering and load factor.
4.	Chaining is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.	Open addressing is used when the frequency and number of keys is known.
5.	Cache performance of chaining is not good as keys are stored using linked list.	Open addressing provides better cache performance as everything is stored in the same table.
6.	Wastage of Space (Some Parts of hash table in chaining are never used).	In Open addressing, a slot can be used even if an input doesn't map to it.
7.	Chaining uses extra space for links.	No links in Open addressing

UNIT-5 (GRAPHS & HASHING)

RECENT TRENDS IN DATA STRUCTURES AND ALGORITHMS

LECTURE NO:-42

1. Decision Intelligence

Whatever a company does, there's no doubt that they need to make good decisions if they are to survive in the corporate world. Data Science and Machine Learning can contribute to this decision-making so that companies and improve their bottom line. Decision Intelligence is basically a composite field containing Artificial Intelligence and Data Science along with concepts of decision-making and managerial science. In simple words, this means that decision-makers like corporate heads, shareholders, etc. can use machine learning algorithms to obtain insights from their data and make the best decisions by leveraging this data. Decision Intelligence is becoming more and more popular because of the edge it provides companies and currently around **33%** of companies use this technology across all sectors. (If it's a company you've heard of, most probably they use Decision intelligence!)

2. Data Stories

Currently, the field of data analytics uses a lot of data visualization dashboards to communicate the data to decision-makers such as shareholders in a company. But now data stories are becoming more and more popular. Would you prefer to just see facts and figures about the data arranged in a dashboard or see a story that shows the data journey for your company? Most of you would pick a good story any day! And that's why data stories are becoming so popular, especially for laypersons who don't have domain-specific knowledge of data analytics. Gartner even predicts that data stories will become the **most popular method** of conveying data insights by 2025. So if you are a good storyteller along with being a good data analyst, you are in luck!

3. Augmented Analytics

Augmented analytics is becoming more and more popular with this market predicted to grow from **$8.4 billion** in 2018 to around **$18.4 billion** globally by 2023. So it is no surprise that it is already heavily used in 2020 with more prospects for growth in 2021. Augmented analytics can enhance data analytics already used by companies by finding a new method of creating, developing, and sharing data analytics with the help of machine learning and artificial intelligence. This means that companies can automate many analytics capabilities such as the creation, analysis, and building of data models. Augmented analytics also makes it much easier to interact with the data and explain the data insights generated which help in data exploration and analysis. This has entirely

changed the face of business intelligence and data analytics wherein users can easily obtain the data, clean it, and then find correlations or patterns.

4. Data Cloud Services

Data can be huge! Some sources even say that more than **2.5 quintillion byte**s of data are created every day in the world (that's 9 zeroes!) While big companies like Google can easily handle their data in warehouses, it's very difficult for smaller companies to manage and store data in order to obtain insights. That's why cloud services are becoming so popular these days for data analytics. Just like Software as a Service, **Data as a Service**(DaaS) is a cloud service that uses cloud computing to provide data storage, data processing, data integration, and data analytics services to companies using a network connection. Hence, Data as a Service can be used by companies to better understand their target audience using data, automate some of their production, create better products according to market demand, etc. In fact, it is estimated that DaaS will be used by around 90% of large companies in order to generate revenue from data by 2020. DaaS is already provided by many service providers such as Microsoft Azure, SAP, etc.

5. X analytics

Data Analytics till now is mostly limited to a single type of data that is in tabular form. Mostly when anyone talks of analytics, the data that comes to mind is rows upon rows of numbers in a spreadsheet. However, a company also has many other forms of data such as video, text, audio, etc. So if companies have to move ahead of their competitors, they need to utilize this type of data as well. That's what X analytics is all about. This can mean video analytics, audio analytics, textual analytics, and so on. A very common example in textual analytics is sentiment analytics wherein companies can analyze the general mood and sentiments of their customers by studying their reviews. Another example is Google video intelligence that is useful in analyzing and classifying objects in videos. In fact, X analytics is becoming so popular that **75% of Fortune 500 companies** might be using it in some shape or form by 2025.

6. Edge Computing

Data is becoming the bread and butter for most companies. However, this data is generated in many places, and mostly, the physical data storage devices for the cloud are far away from where the data is generated. It becomes very costly to transfer this data and also leads to higher data latency. That's where Edge Computing comes in! Edge Computing makes sure that the computational and data storage centers are closer to the edge of the topology where this data is generated or where it is consumed. This is a better alternative than having these storage centers in a central geographical location which is actually thousands of miles from the data being produced or used. Edge Computing ensures that there is no latency in the data that can affect an application's performance and also reduces the money lost in data transmission. And where there

reduction in the loss of money, that technology is sure to get popular. Gartner predicts that **75%** of all the data that is managed by companies will be processed using edge computing as compared to only **10%** in 2018.

7. Blockchain for Data

Security is becoming a much bigger issue for companies than ever before. Data is a goldmine of opportunities but this goldmine can also be hacked and companies could suffer higher losses than they ever have before. So new technologies for data security are becoming critical with Blockchain as one among them. Blockchain is a chain of blocks where these "blocks" constitute digital information that is connected using cryptography and each block references the previous block in the chain. Since the Blockchain is a distributed technology, it is very secure and also transparent. These days many companies are using Blockchain from the available distributed ledgers like Ethereum, R3 Corda, Hyperledger Fabric, Bitcoin, Quorum, etc. which increase the data security and in turn improve data quality because only the important data is protected so much.

MITRC